



Universidad Autónoma de Baja California

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA



Análisis y Diseño de Sistemas.



Punto de Venta Nexo Pay.

Alumnos:

Joshua Osorio O. – 1293271

Andrés Eduardo Fabara C. – 1280728

Docente:

Fernando E. Saucedo L.

13/0/2026

ÍNDICE GENERAL

1	Introducción	5
1.1	Sistema por desarrollar.....	5
1.2	Importancia y relevancia.....	5
1.3	Objetivos.	6
2	Justificación	7
2.1	¿Porque se seleccionó este proyecto?	7
2.2	Beneficios esperados.	7
3	Desarrollo	8
3.1	Fundamentos del Análisis y Diseño de Sistemas.....	8
3.2	Lenguaje Unificado de Modelado (UML).....	11
3.3	Arquitecturas del sistema	33
3.4	Codiseño Hardware-Software.....	34
4	Implementación y Pruebas (Opcional, si aplica).....	38
5	Conclusiones	71
5.1	Reflexión sobre el desarrollo del proyecto.	71
5.2	Lecciones aprendidas.....	71
5.3	Posibles mejoras futuras.	72
6	Bibliografía.....	72

ÍNDICE DE TABLAS

Tabla 1: CU1 Inicio de sesión	14
Tabla 2: CU2 Realizar Venta.....	15
Tabla 3: CU3 Gestionar Devoluciones	16
Tabla 4: CU4 Descuentos y promociones	17
Tabla 5: CU5 Gestionar inventario.	18
Tabla 6: CU6 Procesar pago.....	19
Tabla 7: división del sistema.	35
Tabla 8: Asignación de tareas.....	35
Tabla 9: Consulta todos los usuarios y que tipo de usuario son.	39
Tabla 11: Consulta todos los detalles de venta y que productos corresponden al detalle de la venta.....	41
Tabla 12: Consulta todos los pagos y a qué venta corresponde.	42
Tabla 15: Plan de pruebas unitarias - Usuario.	45
Tabla 16: Plan de pruebas unitarias - Producto.....	46
Tabla 17: Plan de pruebas unitarias - Ventas.....	47
Tabla 19: PCN Plan de pruebas.....	50
Tabla 20: Tabla de Decisión: Inicio de Sesión.....	50
Tabla 21: Tabla de Decisión: Finalizar Venta.	51
Tabla 22: Tabla de Decisión: Registrar Producto.	51
Tabla 23: Tabla de Decisión: Aplicar Descuento.	51
Tabla 25: PCB Administrador – Iniciar sesión.	52
Tabla 26: PCB Administrador – Alta usuario	53
Tabla 27: : PCB Administrador – Modificar usuario.	54
Tabla 28: PCB Administrador – Baja usuario]	54
Tabla 30: PCB Administrador – Consultar usuario.	55
Tabla 31: PCB Administrador – Modificar cliente.	56
Tabla 32: PCB Administrador – consultar cliente.	57
Tabla 33: PCB cajero – Iniciar sesión.	57
Tabla 34: PCB cajero – Seleccionar producto.	58
Tabla 35: PCB cajero – Consultar precio.	58
Tabla 36: PCB cajero – Registrar venta.	59
Tabla 37: PCB cajero – Finalizar transacción.....	59

Tabla 38: PCB cajero – Generar tíquet.	60
Tabla 39: PCB cliente – Realizar pago.....	61
Tabla 40: PCB cliente – Añadir.	62
Tabla 41: PCB cliente – Solicitar productos.	62
Tabla 42: PCB cliente – Carrito de productos.	63
Tabla 43: PCB cliente – Realizar pago.....	64
Tabla 44: PCB ventas – reporte ventas.....	65
Tabla 45: PCB ventas – generar detalle venta.	66
Tabla 46: PCB ventas – modificar venta.	67
Tabla 47: PCB producto – alta productos.....	68
Tabla 48: PCB producto – Modificar producto.	69
Tabla 49: PCB producto – consultar producto.	70
Tabla 50: PCB producto – Ajustar stock.....	70

ÍNDICE DE FIGURAS

Figura 1: Diagrama de caso de uso del administrador	11
Figura 2: Diagrama de caso de uso del cliente.....	12
Figura 3: Diagrama de caso de uso del cajero	13
Figura 6: Diagrama de entidad y relación.....	20
Figura 7: Diagrama De estado Administrador – Alta de producto	21
Figura 8: Diagrama de estados Administrador - Alta usuario	21
Figura 9: Diagrama de estado cliente - Realizar compra.....	21
Figura 10: Diagrama de estado cajero - Elimina producto	21
Figura 11: Diagrama de despliegue.	22
Figura 12: Diagrama de componentes.	22
Figura 13: Diagrama de clases.	23
Figura 14: Diagrama de interfaz.....	24
Figura 15: Diagrama de secuencia administrador - Inicio de sesión.....	25
Figura 16: Diagrama de secuencia administrador - Cerrar sesión.....	25
Figura 17: Diagrama de secuencia administrador - Administrar productos.....	26
Figura 18: Diagrama de secuencia administrador - Administrar Usuarios.	26
Figura 19: Diagrama de secuencia administrador - Consultar reportes.	27
Figura 20: Diagrama de secuencia administrador - Administrar inventario.	27
Figura 21: Diagrama de secuencia administrador - Administrar clientes.	28
Figura 22: Diagrama se secuencia cajero - Iniciar sesión	28
Figura 23: Diagrama se secuencia cajero - Finalizar transacción.....	29
Figura 24: Diagrama se secuencia cajero - Calcular precios.....	29
Figura 25: Diagrama se secuencia cajero - Seleccionar productos.	30

Figura 26: Diagrama se secuencia cajero - Genear tiket.....	30
Figura 27: Diagrama se secuencia cajero - Registrar venta.	31
Figura 28: Diagrama se secuencia cliente - Soliciar producto.	31
Figura 29: Diagrama se secuencia cliente - Añadir productos.....	32
Figura 30: Diagrama se secuencia cliente - Solicitar producto.	32
Figura 31: Diagrama se secuencia cliente - Solicitar productos en APP.....	33

1 Introducción

1.1 Sistema por desarrollar.

NexoPay es un sistema de Punto de Venta (POS, por sus siglas en inglés Point of Sale) diseñado para automatizar y centralizar las operaciones comerciales de pequeños y medianos negocios. El sistema integra en una sola plataforma las funciones de registro de ventas, gestión de inventario, administración de usuarios y generación de reportes, eliminando la necesidad de procesos manuales que son propensos a errores.

Está dirigido a pequeños y medianos negocios que requieren automatizar su proceso de ventas de manera eficiente y confiable.

Es la combinación de software y hardware que ayuda a cobrar, registrar ventas y controlar el negocio.

1.2 Importancia y relevancia.

En el contexto actual, la digitalización de los procesos comerciales es una necesidad fundamental para cualquier negocio que busque competir en el mercado. Un sistema POS como NexoPay responde directamente a esta necesidad al proporcionar:

Eficiencia operativa: La automatización del proceso de ventas reduce considerablemente el tiempo de atención al cliente y minimiza los errores en el cálculo de totales y el manejo de cambios.

Control de inventario en tiempo real: Cada venta registrada actualiza automáticamente el stock disponible, permitiendo al administrador conocer en todo momento el estado del inventario y tomar decisiones oportunas de reabastecimiento.

Trazabilidad de operaciones: Toda transacción queda registrada en la base de datos, lo que permite auditar el historial de ventas y detectar inconsistencias o irregularidades.

Toma de decisiones basada en datos: Los reportes generados por el sistema ofrecen información valiosa sobre el comportamiento de las ventas, los productos más vendidos y el desempeño del negocio.

Seguridad por roles: El sistema implementa control de acceso diferenciado, asegurando que cada usuario únicamente pueda ejecutar las acciones que le corresponden según su rol.

Desde una perspectiva académica, NexoPay representa la aplicación práctica de los principios de la Programación Orientada a Objetos (POO), demostrando cómo conceptos como herencia, encapsulamiento y polimorfismo se traducen en soluciones de software funcionales y organizadas.

1.3 Objetivos.

Generales

Desarrollar un sistema POS funcional, organizado y correctamente estructurado en Java, aplicando los principios fundamentales de la Programación Orientada a Objetos, que permita gestionar las operaciones de ventas, inventario y administración de un pequeño o mediano comercio de manera eficiente, confiable y fácil de mantener.

Específicos

- Implementar un sistema de autenticación por roles que diferencie las acciones disponibles para el cajero y el administrador, garantizando la seguridad y el control de acceso al sistema.
- Diseñar e implementar el módulo de ventas que permita al cajero seleccionar productos, calcular totales automáticamente, procesar pagos y generar tickets de compra.
- Construir el módulo de gestión de productos e inventario que permita al administrador realizar altas, bajas, modificaciones y consultas, manteniendo el stock actualizado tras cada transacción.
- Desarrollar el módulo de gestión de usuarios y clientes, permitiendo al administrador registrar, modificar y consultar la información de cajeros y clientes del sistema.
- Implementar el módulo de reportes que consolide la información de ventas y presente al administrador estadísticas útiles para la toma de decisiones.

- Aplicar correctamente los conceptos de herencia, encapsulamiento y polimorfismo en la estructuración de las clases del sistema, manteniendo un código limpio, modular y bien documentado.
- Integrar el sistema con una base de datos relacional mediante consultas SQL, garantizando la persistencia y consistencia de todos los datos generados por las operaciones del negocio.

2 Justificación

2.1 ¿Porque se seleccionó este proyecto?

La selección de un sistema POS como proyecto académico responde a múltiples criterios que lo convierten en un caso de estudio ideal para la materia de Programación Orientada a Objetos:

Aplicabilidad real: Los sistemas POS son utilizados diariamente por miles de negocios en México y el mundo. Desarrollar uno desde cero permite comprender cómo el software que se estudia en el aula tiene un impacto directo y tangible en la vida cotidiana. Esta conexión con la realidad aumenta la motivación y facilita la comprensión de los conceptos.

Riqueza en conceptos de POO: El dominio del problema de un punto de venta involucra naturalmente entidades como *Usuario*, *Producto*, *Venta*, *Cliente*, *Pago*, que se relacionan entre sí de formas complejas. Esta riqueza de relaciones permite practicar herencia (por ejemplo, *Cajero* y *Administrador* como subclases de *Usuario*, *encapsulamiento* (*atributos privados con accesorios y mutadores*), y *polimorfismo* (*comportamientos distintos según el tipo de usuario*).

2.2 Beneficios esperados.

El proyecto NexoPay ofrece beneficios que abarcan tanto el ámbito académico como el práctico, generando un valor integral para sus desarrolladores y usuarios finales. Desde la perspectiva educativa, permite consolidar los fundamentos de la programación orientada a objetos mediante su implementación en un entorno funcional, al tiempo que fomenta habilidades críticas en diseño de software, como el análisis de requerimientos, el modelado UML, la estructuración de clases y la arquitectura por capas. Además, brinda experiencia en el uso de herramientas profesionales incluyendo entornos de desarrollo integrados, control de versiones y gestores de bases de datos y promueve el trabajo colaborativo, la distribución de roles y la coordinación efectiva entre equipos. En cuanto a los beneficios prácticos, el sistema automatiza el registro de ventas y el cálculo de totales, lo que reduce significativamente el tiempo de atención al cliente en el punto de venta y elimina errores humanos comunes en el manejo de efectivo y el cálculo de cambios. Asimismo, ofrece visibilidad en tiempo real del inventario, disminuyendo el riesgo de desabastecimiento de productos clave; genera reportes que permiten identificar tendencias de ventas para optimizar la gestión del negocio; y centraliza toda la información en una base de datos estructurada, facilitando la recuperación y el análisis de datos históricos.

3 Desarrollo

3.1 Fundamentos del Análisis y Diseño de Sistemas

Descripción del problema y objetivos del sistema.

Los pequeños y medianos comercios enfrentan frecuentemente problemas de gestión derivados de la ausencia de herramientas digitales: registros de ventas imprecisos, dificultad para conocer el estado real del inventario, falta de información consolidada para la toma de decisiones y dependencia de procesos manuales que son lentos y propensos a errores.

Esta situación limita la eficiencia operativa y dificulta el crecimiento del negocio.

NexoPay surge como respuesta a esta problemática. El sistema automatiza el proceso de ventas, garantizando que cada transacción quede registrada con precisión, que el inventario se actualice de forma inmediata y que la información esté siempre disponible para quien la necesite. El objetivo central es transformar un proceso comercial manual y disperso en un flujo de trabajo digital, centralizado y confiable.

Desde el punto de vista técnico, el problema se traduce en la necesidad de diseñar un sistema que:

- Gestione la autenticación y el control de acceso por roles de usuario.
- Registre y almacene transacciones de venta de forma persistente.
- Mantenga la consistencia del inventario ante cualquier operación de venta.
- Proporcione información organizada a través de reportes.
- Sea fácil de usar, mantener y extender por parte de desarrolladores futuros.

Identificación de actores y partes interesadas.

Cajero

El cajero es el usuario encargado de realizar las ventas.

Acciones que puede realizar:

- Iniciar sesión en el sistema.
- Cerrar sesión.
- Registrar nuevas ventas.
- Seleccionar productos.
- Consultar precios de productos.
- Generar tickets de compra.
- Finalizar transacciones.
- Consultar inventario (limitado).

No puede modificar productos ni acceder a configuraciones avanzadas.

Administrador

El administrador tiene control total del sistema.

Acciones que puede realizar:

- Iniciar sesión en el sistema.
- Cerrar sesión.
- Administrar productos (A, M, C)
- Administrar usuarios (A, B, M, C)
- Administrar inventario (A, M, C)
- Administrar cliente (A, M, C).
- Consultar reportes de ventas.
- Supervisar operaciones.

Nomenclatura: A: Altas B: Bajas M: Modificaciones C: Consultas

Cliente

El cliente interactúa indirectamente con el sistema.

Acciones relacionadas:

- Registrar su compra
- Solicitar la compra de productos.
- Recibir ticket de compra.
- Realizar el pago.
- Añadir producto al carrito.
- Añadir propinas.

Recolección y análisis de requerimientos.

Requerimiento Funcional

El sistema POS 'Nexo Pay' debe permitir la gestión integral y automatizada de un punto de venta, facilitando la administración de usuarios (cajeros y administradores), el control de inventario de productos en tiempo real, la ejecución de transacciones de venta con múltiples métodos de pago, y la generación de comprobantes (tickets) y reportes de desempeño; todo esto interactuando de manera eficiente y centralizada con una base de datos relacional.

Requerimiento No Funcional

El sistema 'Nexo Pay' debe ser una aplicación de alta disponibilidad, segura y robusta, capaz de procesar transacciones en tiempo real con tiempos de respuesta mínimos. Debe garantizar la integridad y persistencia de los datos frente a fallos, y ofrecer una

interfaz de usuario gráfica (GUI) intuitiva que minimice la curva de aprendizaje del personal operativo.

Actividades funcionales

- Registrar ventas.
- Generar tickets.
- Gestionar productos.
- Iniciar sesión.
- Calcular totales.
- Actualizar inventario.
- Generar reportes.

Actividades no funcionales

- Solo usuarios autorizados pueden iniciar sesión.
- Solo el administrador puede modificar productos.
- El inventario no puede ser negativo.
- El sistema debe usar base de datos.
- El sistema debe ejecutarse como aplicación de escritorio.
- El sistema debe estar desarrollado en Java.
- Cada venta debe registrarse obligatoriamente.

Principales metodologías aplicadas en el análisis y diseño.

El sistema se diseñó aplicando principios de:

- Análisis orientado a objetos
- Modelado UML
- Arquitectura Monolítica
- Codiseño hardware–software
- Buenas prácticas de ingeniería de requerimientos

Entorno de operación

Aplicación de escritorio
Sistema desarrollado en Java
Uso de base de datos local o remota

Posibles herramientas de desarrollo

Lenguaje de programación: Java
IDE: VSCode
Base de datos: MySQL o SQLite

Interfaz gráfica: Swing

Modelado: UML (casos de uso, diagramas de clases)

3.2 Lenguaje Unificado de Modelado (UML)

Caso de uso.

Representa las interacciones entre los actores del sistema (Cajero, Administrador, Cliente) y las funciones que pueden ejecutar. Su importancia radica en definir el alcance funcional del sistema de forma clara y comprensible para todos los involucrados.

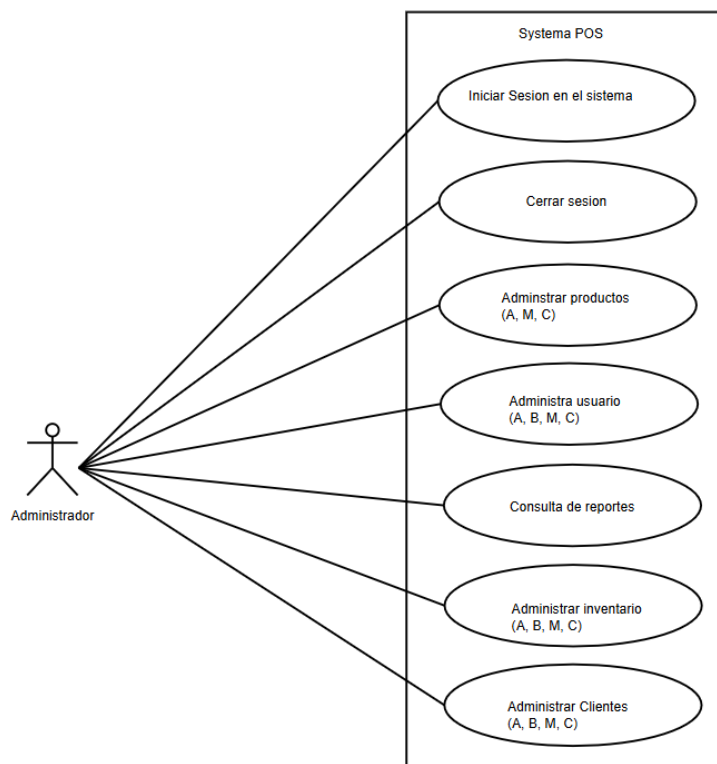


Figura 1: Diagrama de caso de uso del administrador

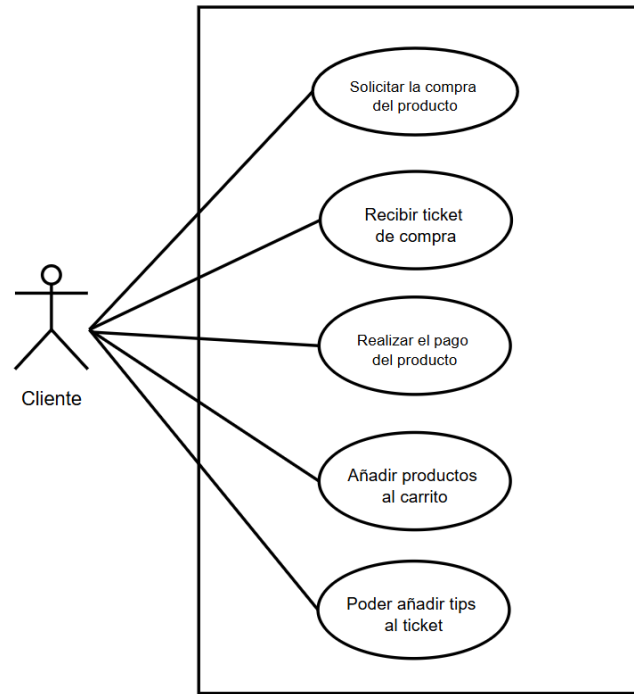


Figura 2: Diagrama de caso de uso del cliente

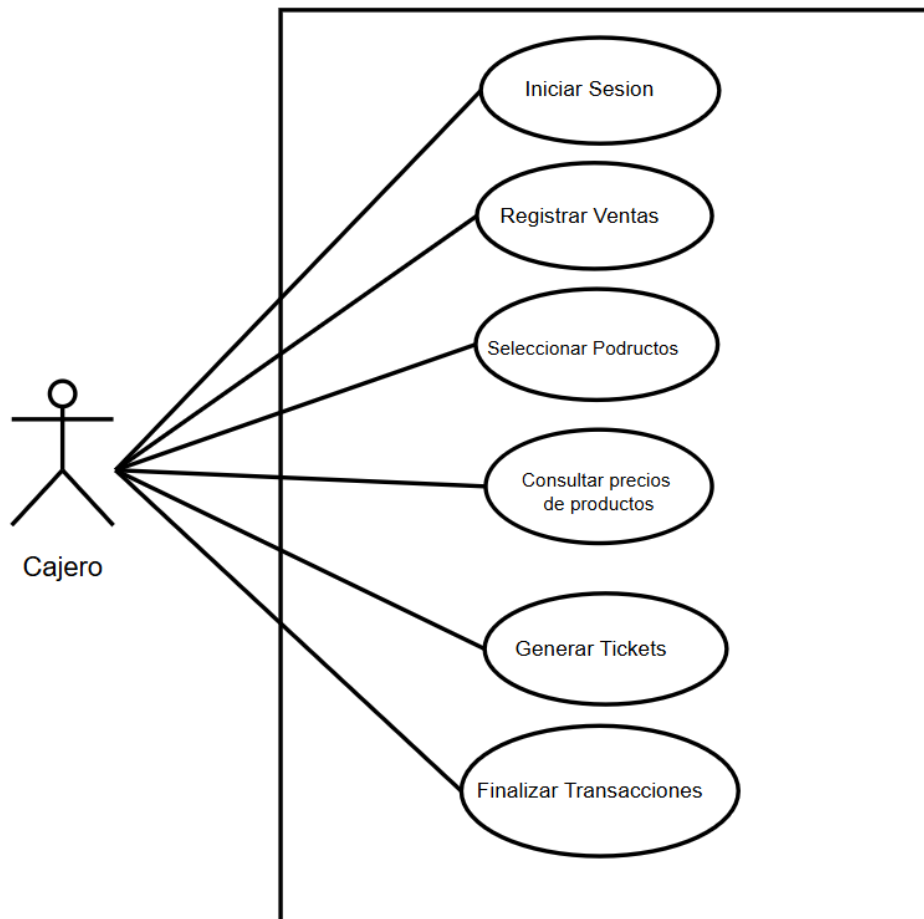


Figura 3: Diagrama de caso de uso del cajero

Detalles de caso de uso.

Describe de forma estructurada cada caso de uso: precondiciones, flujo principal, flujos alternos y postcondiciones. Permite entender con precisión cómo debe comportarse el sistema ante cada acción del usuario.

Tabla 1: CU1 Inicio de sesión

DETALLE DE CASO DE USO — INICIO DE SESIÓN		
Nombre del caso de uso	Inicio de Sesión	
Actor(es) involucrado(s)	Usuario (Cajero, Administrador)	
	Sistema de Autenticación	
Descripción	Permite a los usuarios autenticarse en el sistema POS ingresando sus credenciales para acceder a las funcionalidades según su rol.	
Precondiciones	El usuario debe estar registrado en el sistema.	
	El sistema POS debe estar en funcionamiento.	
Flujo principal (normal)	El usuario accede a la pantalla de inicio de sesión del POS.	
	El usuario ingresa su nombre de usuario y contraseña.	
	El sistema valida las credenciales.	
	Si la validación es exitosa, el sistema concede acceso y muestra el panel principal según el rol.	
Flujos alternativos y excepciones	a. Credenciales incorrectas:	
		El sistema detecta que el usuario o contraseña son incorrectos.
		Se muestra un mensaje de error y se solicita reingresar los datos.
		Se permite hasta 3 intentos antes de bloquear el acceso temporalmente.
	b. Usuario bloqueado:	
		El sistema bloquea la cuenta tras múltiples intentos fallidos.
		Se notifica al administrador para desbloquear la cuenta.
Postcondiciones	El usuario accede al sistema con las funcionalidades habilitadas según su rol.	
	El sistema registra la hora y fecha de inicio de sesión en el log de auditoría.	
Reglas de negocio	La contraseña debe tener mínimo 8 caracteres, una mayúscula y un número.	
	Se permite un máximo de 3 intentos fallidos antes del bloqueo temporal.	
	Cada sesión debe expirar automáticamente tras 30 minutos de inactividad.	

Tabla 2: CU2 Realizar Venta

DETALLE DE CASO DE USO — REALIZAR VENTA		
Nombre del caso de uso	Realizar Venta	
Actor(es) involucrado(s)	Cajero	
	Cliente	
	Sistema de Inventario	
Descripción	Permite al cajero registrar los productos adquiridos por el cliente, calcular el total y procesar el cobro de manera eficiente.	
Precondiciones	El cajero debe haber iniciado sesión en el sistema.	
	Los productos deben estar registrados en el catálogo con precio.	
	La caja debe estar abierta para operar.	
Flujo principal (normal)	El cajero inicia una nueva transacción de venta.	
	El cajero escanea o busca cada producto manualmente.	
	El sistema agrega cada producto a la lista de venta y actualiza el subtotal.	
	El cajero confirma los productos y procede al cobro.	
	El sistema calcula el total incluyendo impuestos.	
	El cajero selecciona el método de pago y procesa el cobro.	
	El sistema genera el ticket/factura y actualiza el inventario.	
Flujos alternativos y excepciones	a. Producto no encontrado:	
		El sistema no encuentra el código de barras escaneado.
		El cajero puede buscar el producto por nombre o código manual.
		Si no existe, se notifica al administrador para agregar el producto.
	b. Stock insuficiente:	
		El sistema detecta que no hay suficiente stock del producto.
		Se muestra un mensaje de advertencia al cajero.
		El cajero informa al cliente y ajusta la cantidad o elimina el producto.
	c. Cancelar venta:	
		El cajero puede cancelar la venta en cualquier momento antes del cobro.
		El sistema anula la transacción y no modifica el inventario.
Postcondiciones	La venta queda registrada en el sistema con todos sus detalles.	
	El inventario se actualiza descontando los productos vendidos.	
	Se genera el comprobante de pago para el cliente.	
Reglas de negocio	No se puede realizar una venta sin al menos un producto agregado.	
	El sistema debe aplicar automáticamente los impuestos configurados (IVA).	
	Toda venta debe tener asociado un cajero responsable.	
	El ticket debe emitirse inmediatamente después de confirmar el pago.	

Tabla 3: CU3 Gestionar Devoluciones

DETALLE DE CASO DE USO — GESTIONAR DEVOLUCIONES	
Nombre del caso de uso	Gestionar Devoluciones
Actor(es) involucrado(s)	Cajero
	Cliente
Descripción	Permite procesar la devolución de productos adquiridos por el cliente, ya sea por defecto, insatisfacción u otro motivo válido, emitiendo el reembolso correspondiente.
Precondiciones	El cajero o administrador debe haber iniciado sesión en el sistema.
	La venta original debe estar registrada en el sistema.
	El cliente debe presentar el comprobante de compra.
Flujo principal (normal)	El cajero accede al módulo de devoluciones.
	El cajero busca la venta original por número de ticket o fecha.
	El sistema muestra el detalle de la venta original.
	El cajero selecciona el o los productos a devolver e indica el motivo.
	El administrador autoriza la devolución (si aplica política de autorización).
	El sistema procesa el reembolso según el método de pago original.
	El sistema actualiza el inventario sumando los productos devueltos.
	Se genera el comprobante de devolución.
Flujos alternativos y excepciones	a. Ticket no encontrado:
	El sistema no encuentra la venta con los datos proporcionados.
	El administrador autoriza la devolución (si aplica política de autorización).
	b. Devolución fuera de plazo:
	El sistema detecta que la compra supera el período de devolución permitido.
	Se notifica al cajero y se requiere autorización del administrador para continuar.
	c. Reembolso en crédito:
	Si no es posible reembolsar en efectivo, se genera un crédito a favor del cliente.
Postcondiciones	El sistema registra el crédito vinculado al cliente para uso futuro.
	La devolución queda registrada en el sistema con motivo y responsable.
	El inventario se actualiza con los productos devueltos.
Reglas de negocio	El cliente recibe el reembolso o crédito correspondiente.
	Las devoluciones tienen un plazo máximo de 30 días desde la compra.
	Se notifica al cajero y se requiere autorización del administrador para continuar.
	El motivo de devolución es obligatorio para registrar la transacción.
	Los productos devueltos deben pasar revisión antes de reintegrarse al inventario.

Tabla 4: CU4 Descuentos y promociones

DETALLE DE CASO DE USO — APLICAR DESCUENTOS Y PROMOCIONES		
Nombre del caso de uso	Aplicar Descuentos y Promociones	
Actor(es) involucrado(s)	Cajero	
	Sistema de Promociones	
Descripción	Permite aplicar descuentos manuales o promociones automáticas configuradas en el sistema sobre los productos o el total de la venta.	
Precondiciones	El cajero debe tener una venta activa en curso.	
	Los descuentos y promociones deben estar configurados en el sistema.	
	El cajero debe tener permisos para aplicar descuentos (según su rol).	
Flujo principal (normal)	El cajero accede a la opción de descuentos durante la venta activa.	
	El sistema muestra las promociones activas aplicables automáticamente.	
	El sistema aplica las promociones automáticas que correspondan.	
	Si se requiere descuento manual, el cajero ingresa el porcentaje o monto.	
	El sistema valida que el descuento esté dentro de los límites permitidos.	
	El sistema recalcula el total con el descuento aplicado.	
	El descuento queda registrado en el ticket de venta.	
Flujos alternativos y excepciones	a. Descuento supera el límite permitido:	
		El sistema detecta que el descuento manual excede el porcentaje máximo.
		Se solicita autorización del administrador para continuar.
		El administrador ingresa sus credenciales para aprobar el descuento.
	b. Promoción no aplicable:	
		El sistema detecta que los productos no cumplen las condiciones de la promoción.
		Se notifica al cajero que la promoción no aplica para los artículos seleccionados.
Postcondiciones	El descuento o promoción queda registrado en la transacción de venta.	
	El total de la venta refleja el precio final con descuento.	
	El sistema almacena el tipo de descuento y el responsable de aplicarlo.	
Reglas de negocio	Los cajeros solo pueden aplicar descuentos de hasta el 10%; montos mayores requieren administrador.	
	No se pueden combinar dos promociones del mismo tipo en una misma venta.	
	Los descuentos no pueden hacer que el precio sea menor al costo del producto.	
	Todo descuento manual debe tener un motivo registrado.	

Tabla 5: CU5 Gestionar inventario.

DETALLE DE CASO DE USO — GESTIONAR INVENTARIO		
Nombre del caso de uso	Gestionar Inventario	
Actor(es) involucrado(s)	Administrador	
	Sistema de Inventario	
Descripción	Permite registrar, actualizar, consultar y controlar el stock de productos disponibles en el punto de venta, incluyendo altas, bajas y ajustes de inventario.	
Precondiciones	El usuario debe haber iniciado sesión con rol de Administrador o administrador.	
	Los productos deben tener categoría, código y precio asignados.	
Flujo principal (normal)	El administrador accede al módulo de inventario.	
	El administrador selecciona la operación: agregar, editar, eliminar o ajustar.	
	Para agregar: ingresa código, nombre, categoría, precio, costo y stock inicial.	
	El sistema valida que no exista un producto con el mismo código.	
	El sistema guarda los cambios y actualiza el catálogo de productos.	
	El sistema registra el movimiento en el historial con fecha y usuario.	
Flujos alternativos y excepciones	a. Código de producto duplicado:	
		El sistema detecta que el código ingresado ya existe.
		Se muestra un mensaje de error solicitando un código único.
		El administrador corrige el código y reintenta.
	b. Stock por debajo del mínimo:	
		El sistema detecta que el stock alcanzó el nivel mínimo configurado.
		Se genera una alerta automática para reabastecer el producto.
		El administrador puede generar una orden de compra desde el sistema.
	c. Ajuste por diferencia física:	
		El administrador realiza un conteo físico y detecta diferencias.
		Registra el ajuste indicando motivo (merma, robo, error de captura).
		El sistema actualiza el stock con el valor real contado.
Postcondiciones	El catálogo de productos refleja la información actualizada.	
	El stock disponible está sincronizado con los movimientos registrados.	
	Todos los cambios quedan auditados con fecha, hora y usuario responsable.	
Reglas de negocio	Solo usuarios con rol Administrador pueden modificar el inventario.	
	El stock no puede ser negativo; el sistema debe impedirlo.	
	Todo ajuste de inventario debe llevar un motivo obligatorio.	
	Los productos eliminados se desactivan lógicamente, no se borran físicamente.	

Tabla 6: CU6 Procesar pago

DETALLE DE CASO DE USO — PROCESAR PAGO	
Nombre del caso de uso	Procesar Pago
Actor(es) involucrado(s)	Cajero
	Cliente
	Terminal Bancaria
	Sistema de Pagos
Descripción	Permite procesar el pago de una venta mediante diferentes métodos: efectivo, tarjeta de crédito/débito, transferencia o pago mixto.
Precondiciones	Debe existir una venta activa con productos registrados.
	El cajero debe haber confirmado el total de la venta.
	Los métodos de pago deben estar habilitados en la configuración del sistema.
Flujo principal (normal)	El sistema muestra el total a pagar al cajero.
	El cajero selecciona el método de pago (efectivo, tarjeta, transferencia).
	Para pago en efectivo: el cajero ingresa el monto recibido del cliente.
	El sistema calcula el cambio a devolver al cliente.
	Para pago con tarjeta: el sistema se comunica con la terminal bancaria.
	La terminal procesa la transacción y confirma la autorización.
	El sistema registra el pago y genera el comprobante de venta.
Flujos alternativos y excepciones	a. Tarjeta rechazada:
	La terminal bancaria devuelve un rechazo de la transacción.
	El sistema muestra el motivo del rechazo al cajero.
	El cajero ofrece al cliente intentar con otra tarjeta o método de pago.
	b. Pago mixto:
	El cliente desea pagar con dos métodos diferentes.
	El cajero selecciona 'pago mixto' e ingresa el monto para cada método.
	El sistema valida que la suma cubra el total de la venta.
	Se procesan ambos pagos y se genera un único comprobante.
	c. Error en terminal bancaria:
	La terminal no responde o pierde conexión.
	El sistema notifica el error de comunicación al cajero.
	El cajero puede esperar reconexión o solicitar otro método de pago.
Postcondiciones	El pago queda registrado en el sistema con método, monto y fecha.
	La venta queda marcada como pagada y cerrada.
	Se genera el comprobante de pago para el cliente.
Reglas de negocio	El monto recibido no puede ser menor al total de la venta para pago en efectivo.
	Las transacciones con tarjeta requieren autorización de la terminal bancaria.
	El sistema debe registrar el número de autorización bancaria para pagos con tarjeta.
	El cambio máximo en efectivo no puede superar el fondo de caja disponible.

Diagrama de Entidad y relación.

Modela la estructura de la base de datos: entidades, atributos y relaciones entre tablas. Es fundamental para garantizar una persistencia de datos correcta y sin redundancias en MySQL.

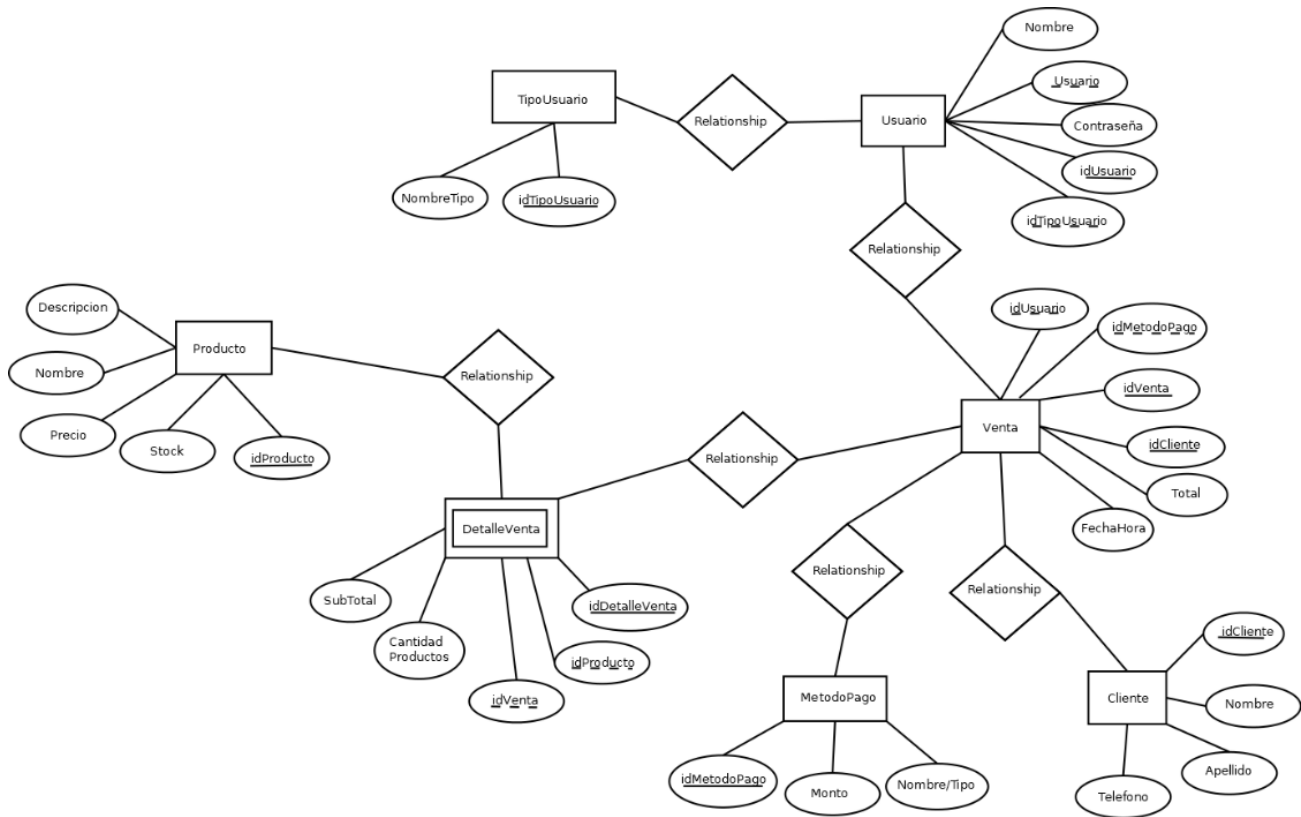


Figura 5: Diagrama de entidad y relación.

Diagrama de estados.

Ilustra los posibles estados de un objeto a lo largo de su ciclo de vida y las transiciones entre ellos. En el POS es útil para modelar el comportamiento de una venta (iniciada, en proceso, finalizada, cancelada).

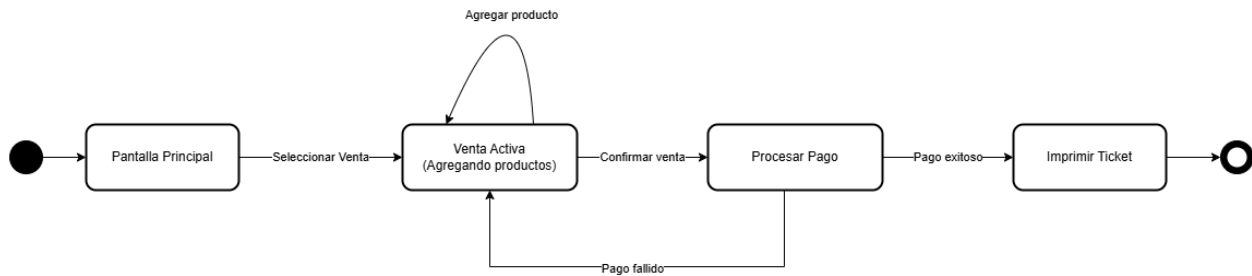


Figura 6: Diagrama De estado Administrador – Alta de producto

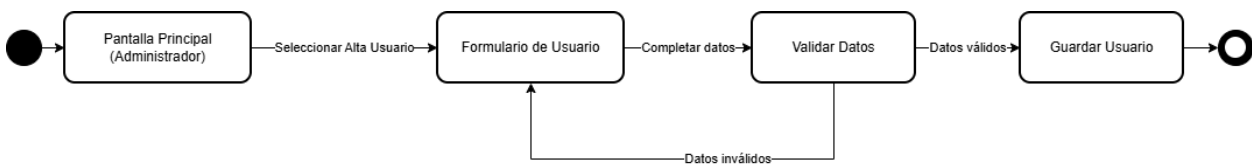


Figura 7: Diagrama de estados Administrador - Alta usuario

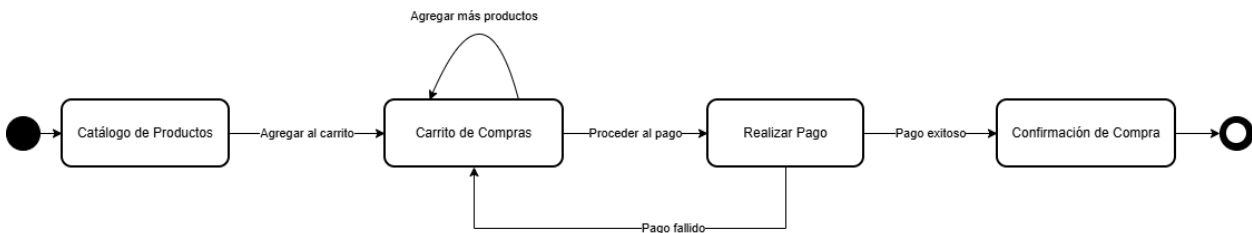


Figura 8: Diagrama de estado cliente - Realizar compra

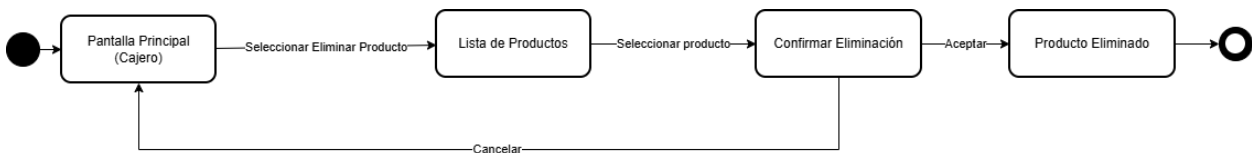


Figura 9: Diagrama de estado cajero - Elimina producto

Diagrama de despliegue.

Representa la distribución física del sistema: hardware, nodos y cómo se conectan los componentes de software sobre ellos. Permite visualizar el entorno de ejecución real del POS.

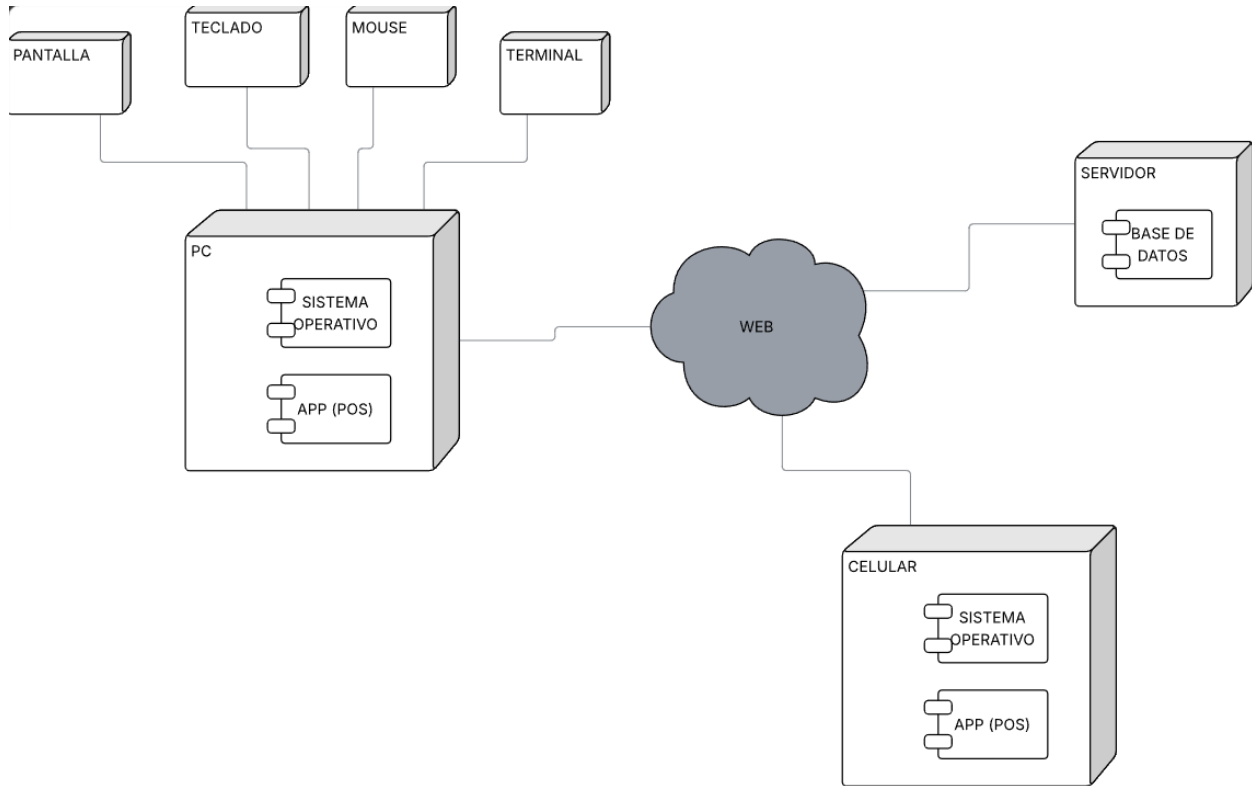


Figura 10: Diagrama de despliegue.

Diagrama de componentes.

Muestra los módulos de software del sistema y sus dependencias. Ayuda a entender la organización interna del proyecto y cómo se relacionan capas como la interfaz, la lógica de negocio y el acceso a datos.



Figura 11: Diagrama de componentes.

Diagrama de clases.

Muestra las clases del sistema, sus atributos, métodos y relaciones (herencia, asociación, dependencia). Es el diagrama central del diseño orientado a objetos y sirve como base directa para la implementación en Java.

UABC_FCQI_ANALISIS_Y_DISEÑO_DE_SISTEMAS.

Diagrama de clases Sistema POS

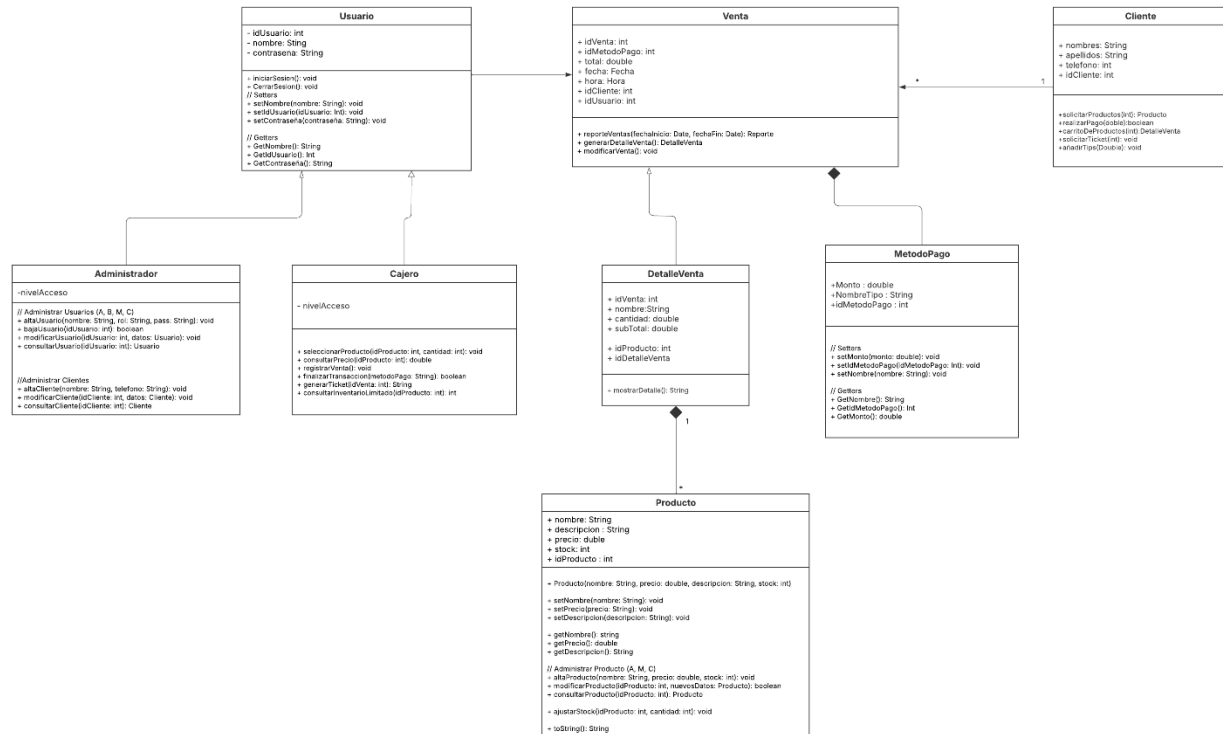


Figura 12: Diagrama de clases.

Diagrama de interfaz.

Representa visualmente las pantallas o menús con los que interactúa el usuario. Permite validar con anticipación la experiencia de uso antes de comenzar la implementación.

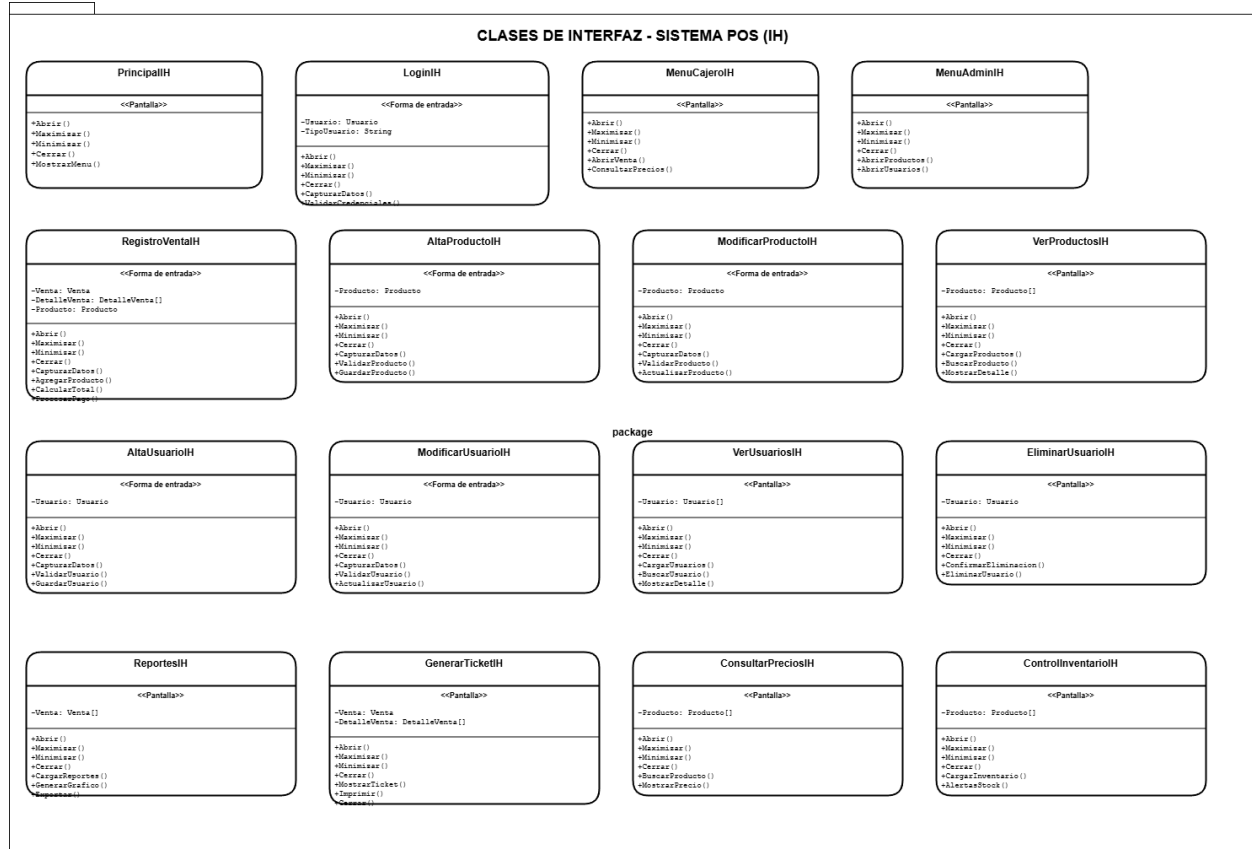


Figura 13: Diagrama de interfaz

Diagrama de secuencias.

Describe el flujo de mensajes entre objetos a lo largo del tiempo para un escenario específico. Es clave para entender cómo colaboran las clases durante operaciones como el registro de una venta.

Administrador

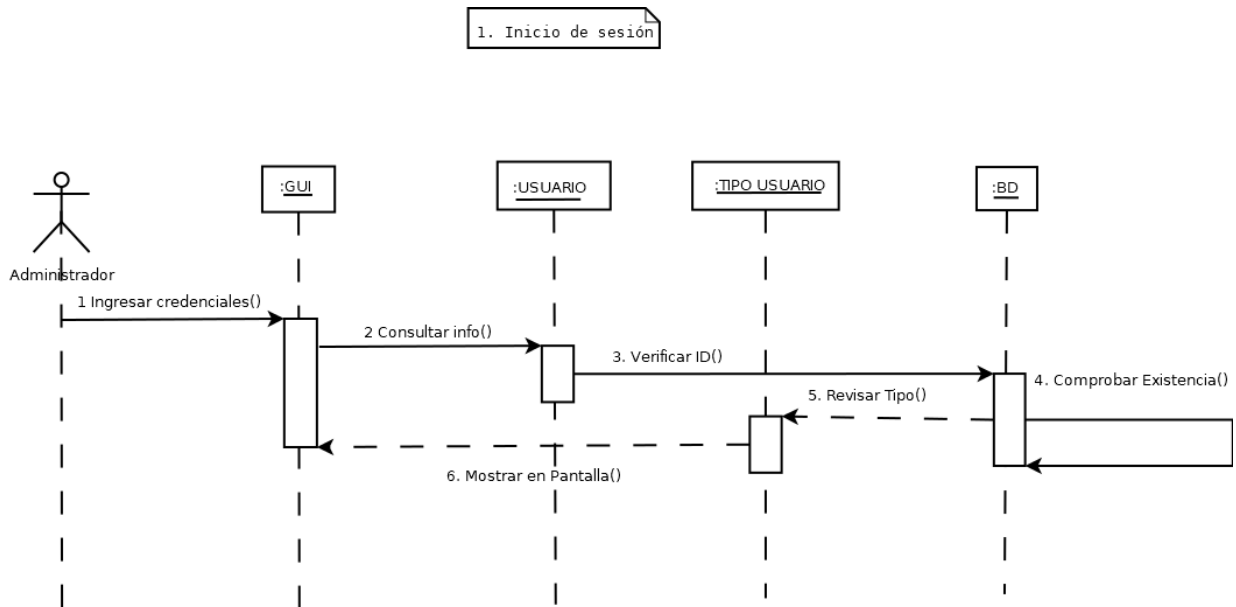


Figura 14: Diagrama de secuencia administrador - Inicio de sesión.

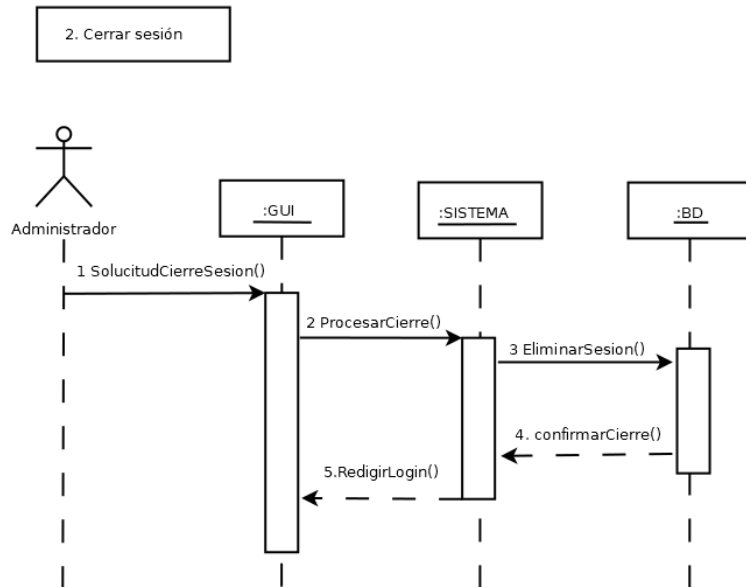


Figura 15: Diagrama de secuencia administrador - Cerrar sesión

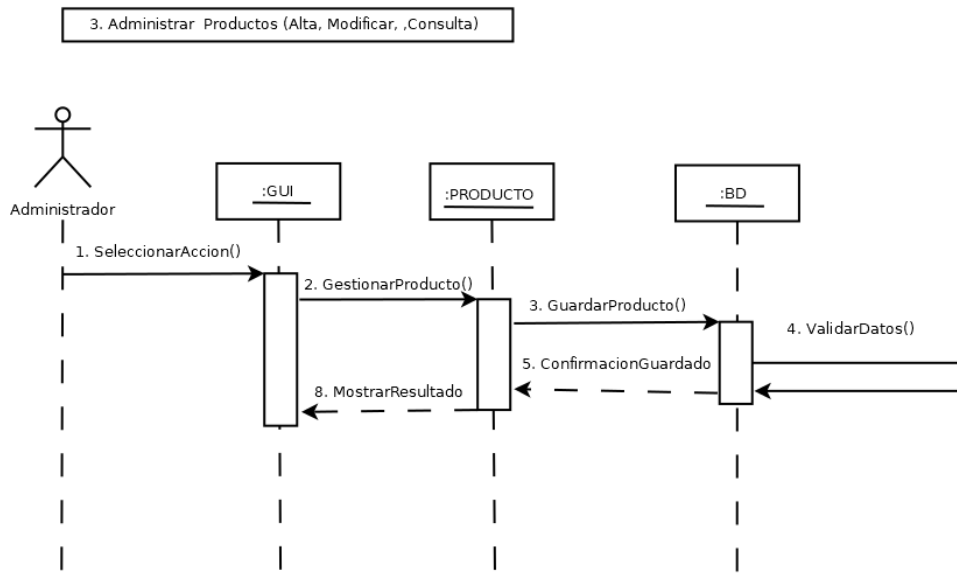


Figura 16: Diagrama de secuencia administrador - Administrar productos

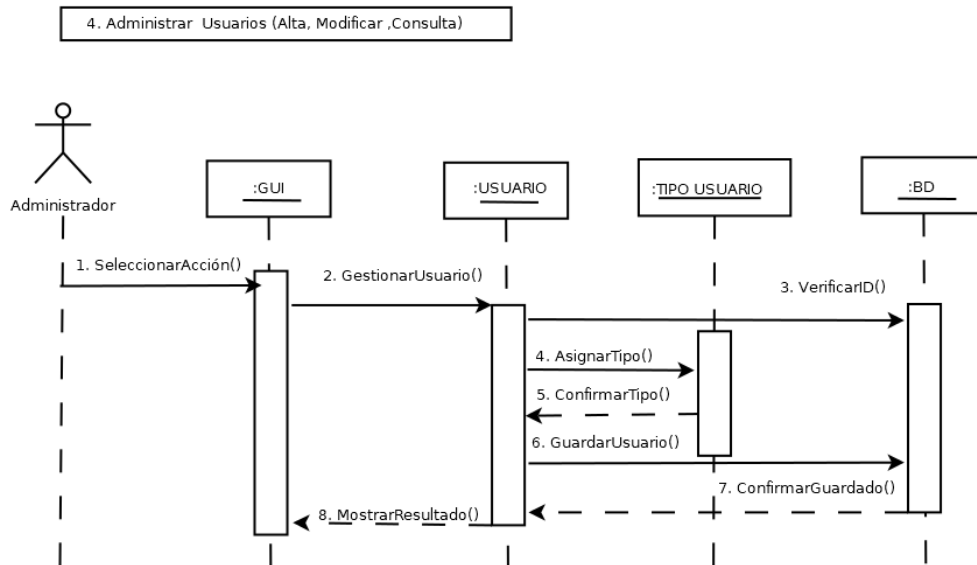


Figura 17: Diagrama de secuencia administrador - Administrar Usuarios.

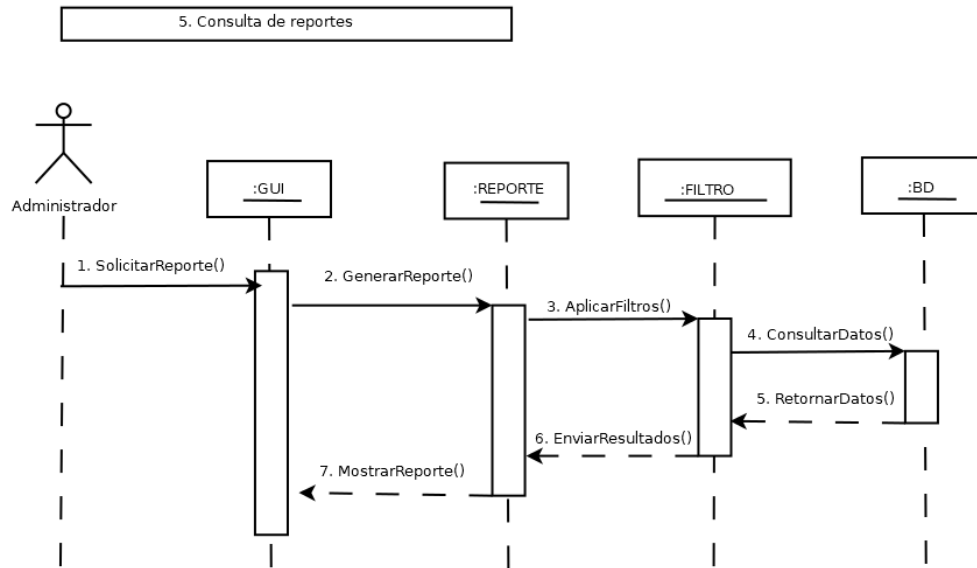


Figura 18: Diagrama de secuencia administrador - Consultar reportes.

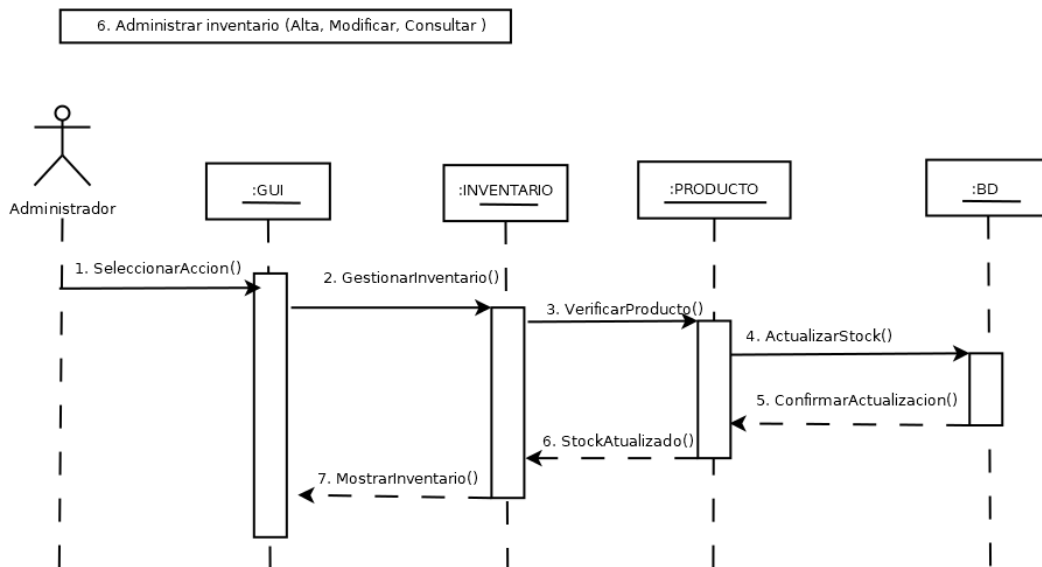


Figura 19: Diagrama de secuencia administrador - Administrar inventario.

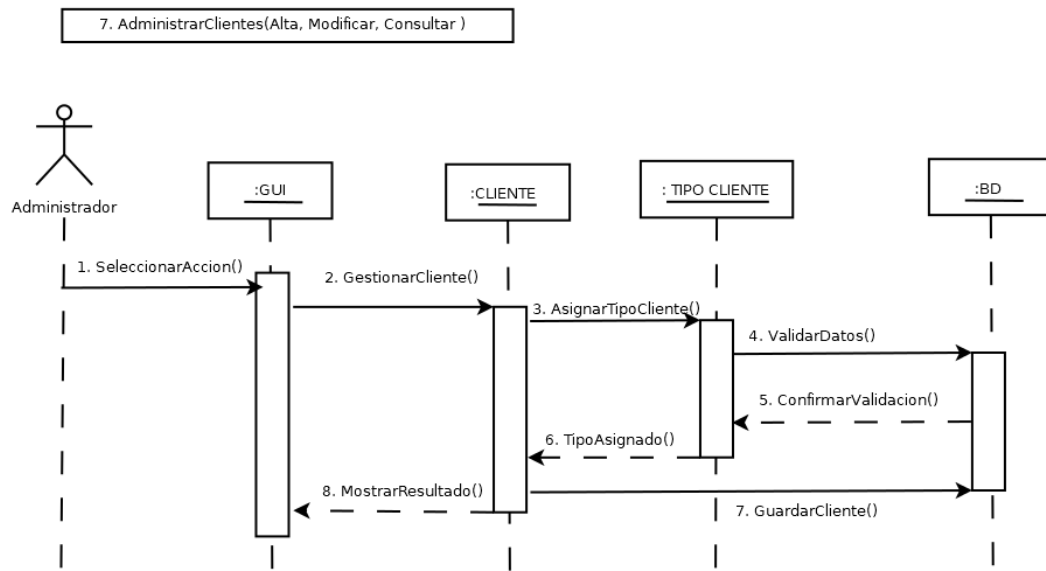


Figura 20: Diagrama de secuencia administrador - Administrar clientes.

Cajero

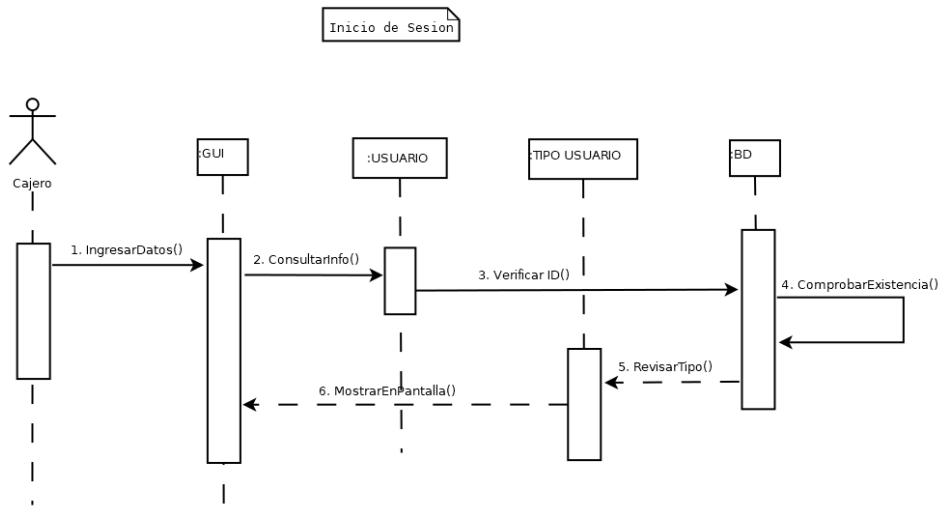


Figura 21: Diagrama de secuencia cajero - Iniciar sesión

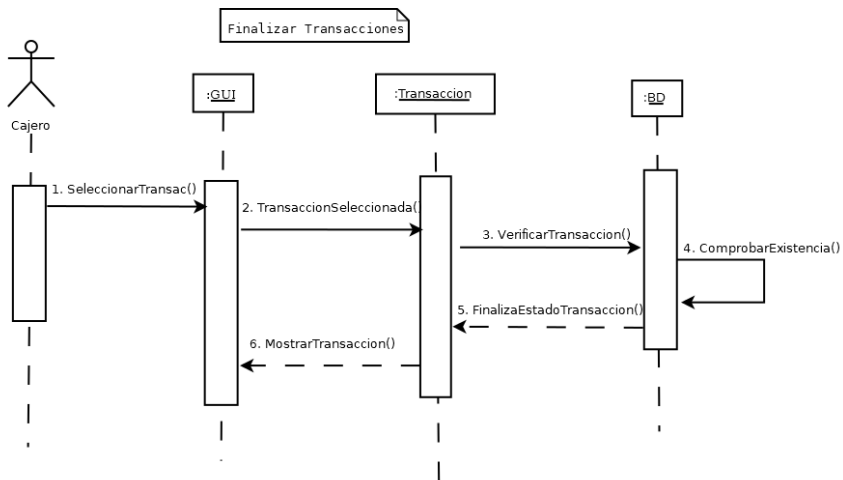


Figura 22: Diagrama de secuencia cajero - Finalizar transacción.

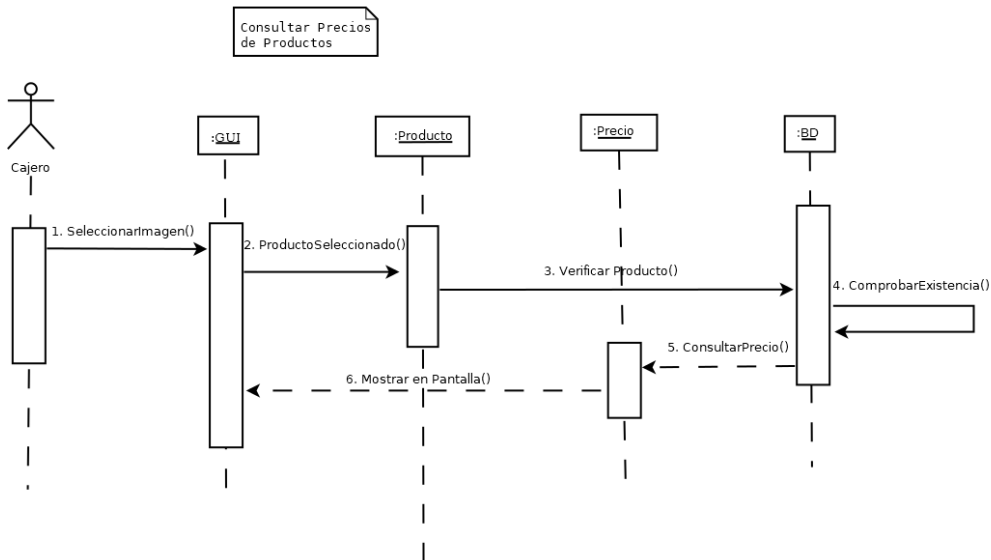


Figura 23: Diagrama de secuencia cajero - Calcular precios.

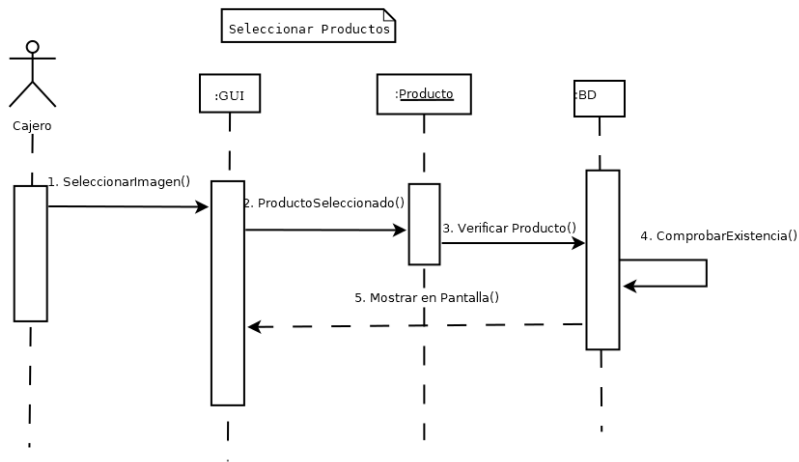


Figura 24: Diagrama de secuencia cajero - Seleccionar productos.

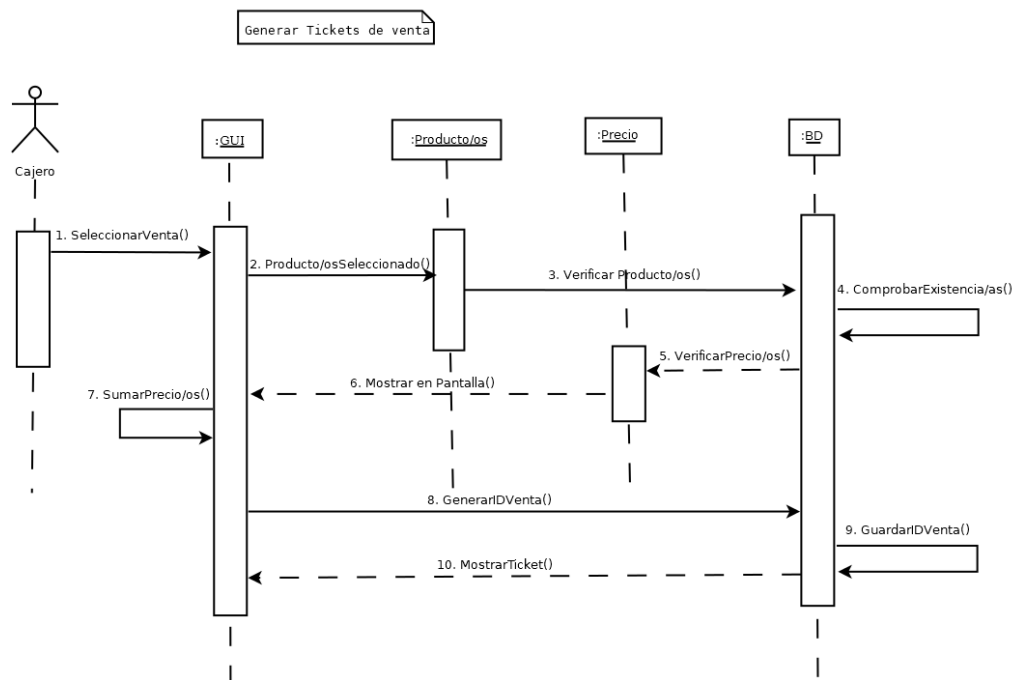


Figura 25: Diagrama de secuencia cajero - Generar ticket..

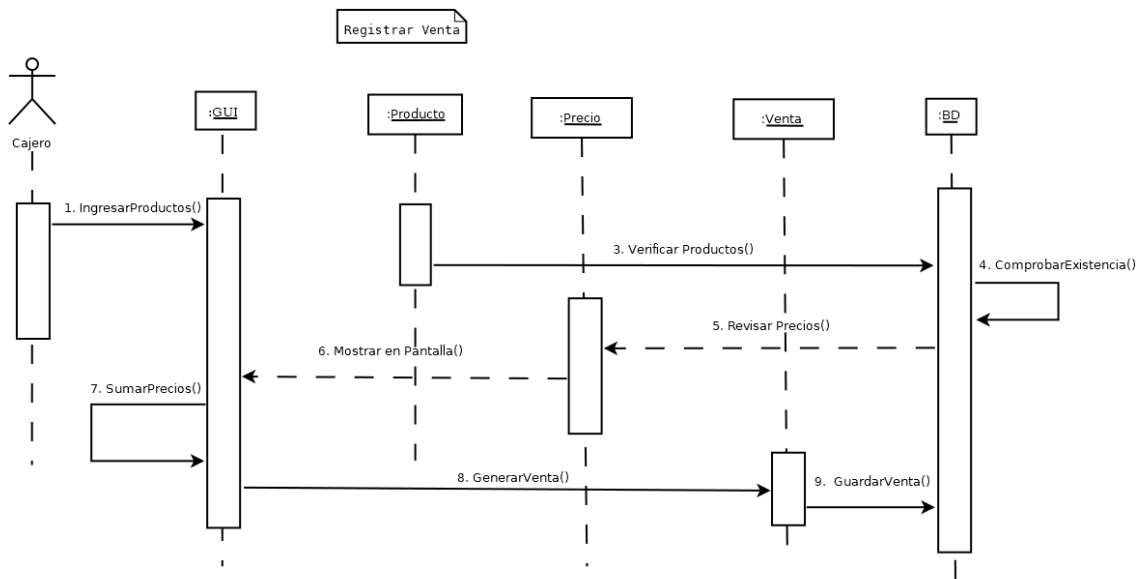


Figura 26: Diagrama se secuencia cajero - Registrar venta.

Cliente

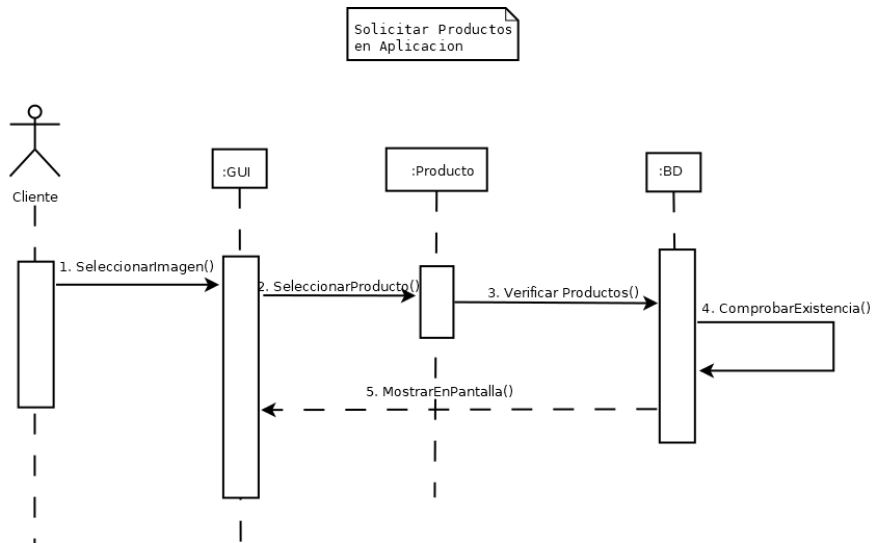


Figura 27: Diagrama se secuencia cliente - Soliciar producto.

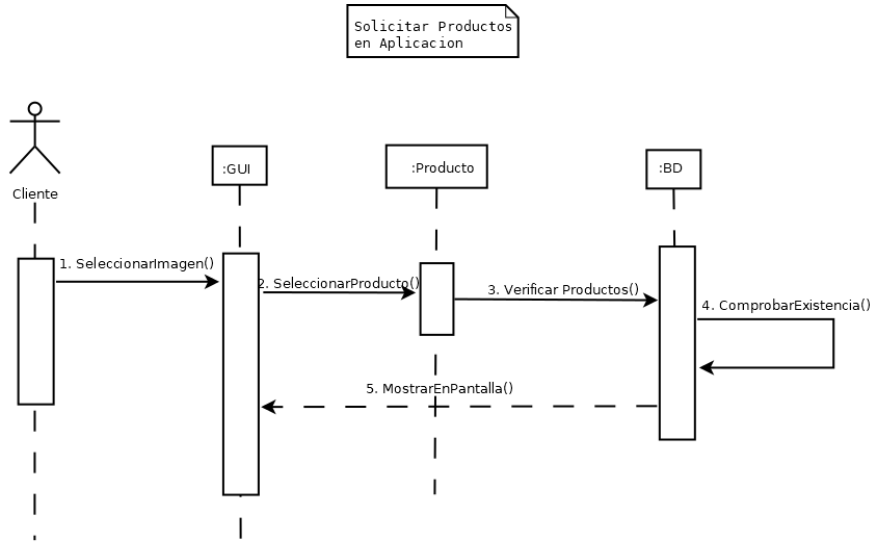


Figura 28: Diagrama de secuencia cliente - Añadir productos

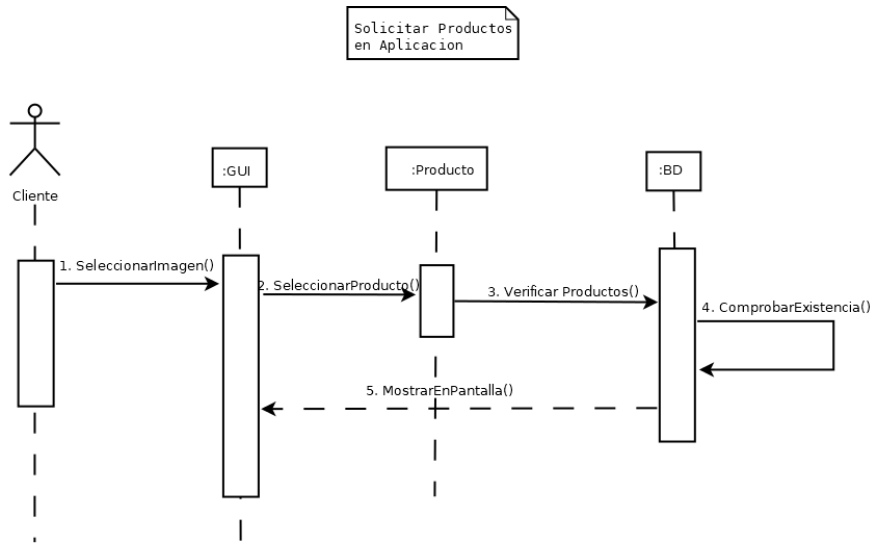


Figura 29: Diagrama de secuencia cliente - Solicitar producto.

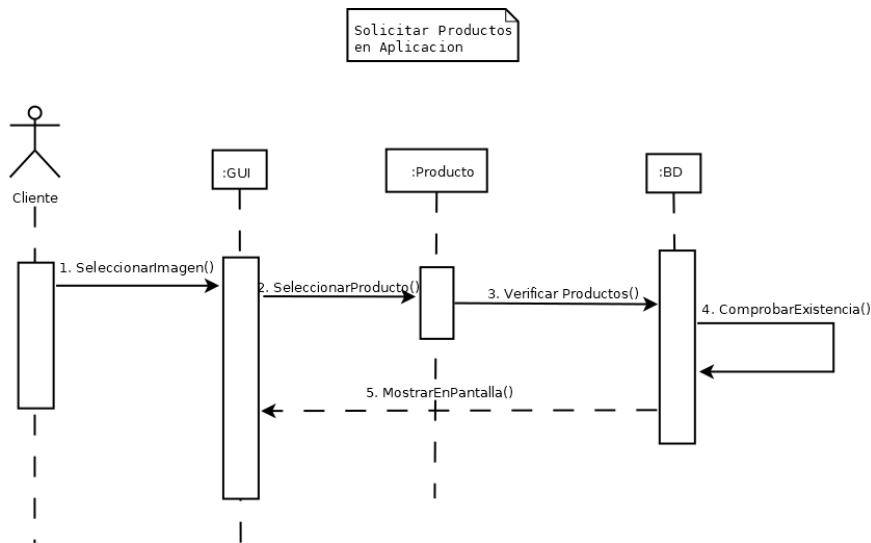


Figura 30: Diagrama de secuencia cliente - Solicitar productos en APP.

3.3 Arquitecturas del sistema

Diseño de la arquitectura del sistema.

Arquitectura Monolítica:

El sistema se construirá como una aplicación autónoma e integrada donde la capa de presentación (interfaz de usuario), la lógica de negocio y la capa de acceso a datos coexisten dentro de la misma base de código. Dependiendo de la implementación de la base de datos, funcionará de manera local o como un sistema Cliente-Servidor clásico conectándose directamente al gestor de base de datos.

Justificación de la elección

- **Naturaleza del proyecto:** Al ser un sistema de tipo transaccional enfocado en una aplicación de escritorio, una arquitectura monolítica es el estándar más eficiente, evitando la sobrecarga innecesaria que tendrían las arquitecturas distribuidas.
- **Herramientas de desarrollo:** El uso del lenguaje Java en conjunto con la biblioteca de interfaz gráfica Swing se adapta naturalmente a la creación de ejecutables monolíticos.
- **Escalabilidad y usuarios:** Está pensado para negocios pequeños y medianos con roles centralizados (cajero, administrador del sistema). Un monolítico facilita la instalación y el despliegue directo en los equipos de estos comercios.

- Gestión de la información: La integración con una base de datos, ya sea local (SQLite) o remota (MySQL) , es directa y robusta dentro de esta arquitectura, permitiendo un manejo rápido para el registro de ventas y el control básico de inventario

Tecnologías y herramientas seleccionadas.

Java 17 LTS: Lenguaje principal del proyecto; estable, amplio soporte académico y orientado a objetos por naturaleza

Entorno de desarrollo (IDE): Visual Studio Code: extensión Pack for Java por comodidad y flexibilidad.

Git y GitHub: Para controlar versiones y trabajo colaborativo.

XAMPP: Para montar los servicios en algún servidor local.

MySQL: Para las bases de datos.

Draw.io: Para los diagramas de clases requeridos para este proyecto.

3.4 Codiseño Hardware-Software

Identificación de Componentes

Para la implementación del sistema POS, se han identificado los siguientes elementos de hardware y software que deben integrarse eficientemente:

Hardware (Periféricos y Terminal)

- Unidad Central de Procesamiento (PC/Laptop): Ejecuta la Máquina Virtual de Java (JVM).
- Escáner de Código de Barras: Dispositivo de entrada USB/HID para identificación de productos.
- Impresora Térmica (ESC/POS): Salida física para tickets de venta.
- Cajón de Dinero: Actuador físico conectado vía RJ11 a la impresora.
- Monitor/Pantalla Táctil: Interfaz visual para el cajero.

Software (Capa Lógica y Datos)

- Lenguaje de Programación: Java (Swing para GUI).
- Gestor de Base de Datos: MySQL (Persistencia de inventario y transacciones).
- Controladores de Dispositivos: Drivers USB-Serial y protocolos ESC/POS.
- Sistema Operativo: Windows o Linux con soporte para JRE.

División del Sistema

Siguiendo la metodología de Codiseño, las funciones se distribuyen entre hardware y software según su naturaleza técnica:

Tabla 7: división del sistema.

Función	Responsable	Justificación
Captura de IDs de producto	Hardware (Escáner)	Lectura óptica inmediata con decodificación integrada.
Cálculo de impuestos y totales	Software (Java)	Lógica aritmética compleja y reglas de negocio dinámicas.
Control de Inventario	Software (MySQL)	Gestión de grandes volúmenes de datos y concurrencia.
Generación de Comprobante	Software + Hardware	El software formatea el ticket; el hardware lo imprime físicamente.
Apertura de Cajón	Hardware (Actuador)	Respuesta mecánica a una señal eléctrica enviada por el software.

Asignación Detallada

Mapa de responsabilidades para asegurar el flujo de datos correcto entre capas:

Tabla 8: Asignación de tareas.

Módulo	Asignación	Descripción Técnica
Módulo de Venta	Java / Swing	Captura eventos del teclado/escáner y actualiza la UI.
Base de Datos	MySQL	Almacena tablas de productos, ventas y usuarios.
Interfaz de Impresión	Driver ESC/POS	Envío de comandos de corte y texto a la impresora térmica.

Métodos de Verificación Sugeridos

- Simulación de Periféricos: Uso de emuladores de puerto serie para validar el envío de comandos de impresión sin hardware físico.
- Pruebas de Integración Escáner-DB: Validar que, al leer un código específico, el software recupere el precio correcto de la base de datos en menos de 100ms.
- Prueba de Flujo Completo: Registro de venta -> Descuento de stock -> Impresión de ticket -> Apertura de cajón.
- Pruebas de Estrés: Simulación de múltiples ventas simultáneas para verificar la estabilidad de la conexión con el servidor de base de datos.

Relación entre software y hardware dentro del sistema.

Refinamiento del Sistema

En el sistema **Nexo Pay**, el codiseño integra tres dimensiones fundamentales: la lógica del software, los componentes de hardware y la experiencia del usuario final.

El refinamiento de este proceso se divide en los siguientes niveles de interacción:

1. Refinamiento de la Interfaz y Experiencia del Usuario (Codiseño UI/UX):

- **Módulo del Cajero:** La interfaz gráfica (GUI) se diseña para depender mínimamente del ratón. Se refina el sistema para que el campo de texto del "ID del Producto" esté siempre enfocado. Así, cuando el cajero utiliza el lector de código de barras, el producto se agrega automáticamente al DetalleVenta y el SubTotal se recalcula en tiempo real sin requerir clics adicionales.
- **Módulo del Administrador:** Se refina la interfaz para mostrar alertas visuales (ej. colores en rojo) cuando el método consultarInventarioLimitado() detecta que un producto tiene un stock crítico, permitiendo la toma de decisiones rápidas para ajustarStock().

2. Refinamiento de la Integración Hardware-Software:

- **Lector de Código de Barras:** Se refina la captura de datos entendiendo que el escáner actúa como un dispositivo de entrada estándar (teclado). El software debe tener "Listeners" (escuchadores de eventos) que detecten la cadena de texto escaneada y ejecuten automáticamente el método seleccionarProducto(idProducto, cantidad) al detectar el salto de línea (Enter) que envía el escáner al final.
- **Impresora Térmica de Tickets:** El método generarTicket(idVenta) no solo devuelve un String, sino que en el refinamiento del codiseño, este texto debe ser formateado en comandos compatibles con impresoras térmicas (ej. estándar ESC/POS) para garantizar cortes de papel automáticos y alineación correcta de los precios y productos.

3. Refinamiento de la Comunicación de Datos:

- **Conexión a la Base de Datos:** Se refina el uso de conectores (como JDBC en el caso de Java) para enlazar la aplicación gráfica con el servidor MySQL (XAMPP).
- **Manejo de Concurrencia:** Si el sistema escala a múltiples cajas, el refinamiento exige que las consultas de actualización (UPDATE en la tabla Producto durante una venta) bloqueen temporalmente el registro para evitar que dos cajeros vendan la misma última unidad de un producto al mismo tiempo, garantizando la integridad de la base de datos.

Diagrama del sistema

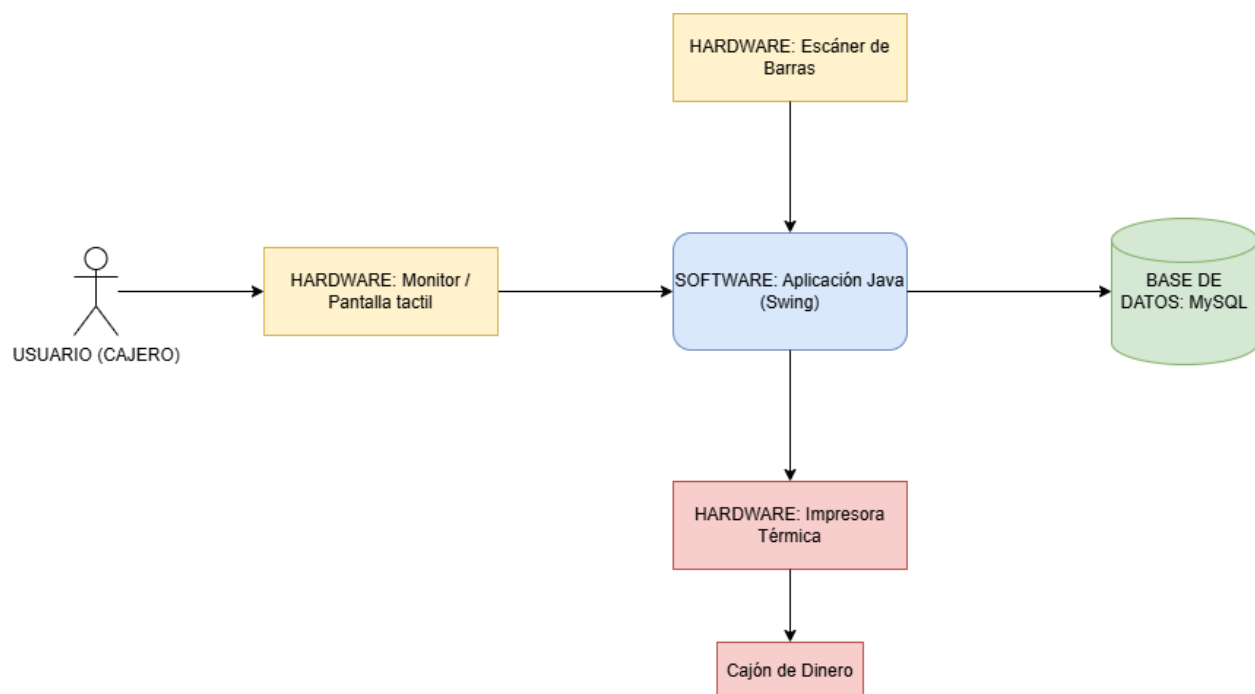


Figura 31: Diagrama del sistema y como se relacionan.

Selección de componentes de hardware.

Los componentes seleccionados son: PC/Laptop como unidad de ejecución de la JVM, escáner de código de barras USB/HID para captura de productos, impresora térmica ESC/POS para emisión de tickets, cajón de dinero conectado vía RJ11 a la impresora, y monitor o pantalla táctil como interfaz del cajero.

Estrategias de integración software-hardware.

La integración se basa en tres estrategias: el escáner actúa como dispositivo HID estándar, por lo que el software lo trata como entrada de teclado sin drivers especiales; la impresora recibe comandos ESC/POS enviados directamente desde Java mediante un driver serial, permitiendo formatear e imprimir tickets sin librerías complejas; y el cajón de dinero se activa

automáticamente mediante una señal eléctrica que la impresora emite al finalizar la impresión del ticket, sin intervención directa del software.

4 Implementación y Pruebas (Opcional, si aplica)

Consultas de bases de datos.

Contiene las sentencias SQL utilizadas para recuperar, insertar y relacionar datos entre tablas. Validan que el modelo relacional responde correctamente a las necesidades del sistema.

Consulta tablas de datos

Muestra los registros almacenados en la base de datos resultado de las operaciones del sistema. Sirve para verificar la integridad y consistencia de la información persistida.

Consulta todos los usuarios y que tipo de usuario son.

Sentencia / Comando utilizada/o:

```
SELECT Usuario.Nombre, tipousuario.NombreTipo  
from Usuario INNER JOIN tipousuario  
ON Usuario.idTipoUsuario=tipousuario.idTipoUsuario;
```

Resultado:

Tabla 9: Consulta todos los usuarios y que tipo de usuario son.

Nombre	Nombre Tipo
Admin General	Administrador
Juan Pérez	Cajero
María García	Cajero
Pedro López	Cajero
Ana Martínez	Cajero
Lucía Sánchez	Cajero
Roberto Díaz	Cajero
Elena Torres	Cajero
Carlos Rodríguez	Encargado de Turno
Miguel Ángel	Encargado de Turno

Consulta todos los usuarios y que tipo de usuario son.

Sentencia o Comando utilizado/a:

SELECT * FROM producto;

Resultado:

UABC_FCQI_ANALISIS_Y_DISEÑO_DE_SISTEMAS.

Tabla 10: Consulta todos los usuarios y que tipo de usuario son.

← T →				▼	idProducto	Nombre	Descripcion	Precio	Stock
	Editar	Copiar	Borrar		1	Coca-Cola 600ml	Refresco de cola original	18.50	50
	Editar	Copiar	Borrar		2	Sabritas Sal 45g	Papas fritas con sal	17.00	40
	Editar	Copiar	Borrar		3	Gansito Marínela	Pastelito relleno	15.50	30
	Editar	Copiar	Borrar		4	Leche Entera 1L	Lácteo pasteurizado	26.00	20
	Editar	Copiar	Borrar		5	Pan Blanco Grande	Pan de caja	45.00	15
	Editar	Copiar	Borrar		6	Agua Ciel 1L	Agua purificada	12.00	100
	Editar	Copiar	Borrar		7	Café Americano 12oz	Café caliente de máquina	22.00	200
	Editar	Copiar	Borrar		8	Vikingo Clásico	Hot dog de la casa	25.00	24
	Editar	Copiar	Borrar		9	Hielo Bolsa 5kg	Cúbitos de hielo	35.00	10
	Editar	Copiar	Borrar		10	Cerveza Six Pack	Lata 355ml	98.00	15

Consulta todos los detalles de venta y que productos corresponden al detalle de la venta.

Sentencia o Comando utilizado/a:

```
SELECT DetalleVenta.idVenta, Producto.Nombre, DetalleVenta.Cantidad, DetalleVenta.SubTotal
FROM DetalleVenta INNER JOIN Producto
ON DetalleVenta.idProducto=Producto.idProducto;
```

Resultado:

UABC_FCQI_ANALISIS_Y_DISEÑO_DE_SISTEMAS.

Tabla 11: Consulta todos los detalles de venta y que productos corresponden al detalle de la venta.

idVenta	Nombre	Cantidad	SubTotal
1	Coca-Cola 600ml	1	18.50
3	Coca-Cola 600ml	1	18.50
1	Coca-Cola 600ml	1	18.50
3	Coca-Cola 600ml	1	18.50
10	Coca-Cola 600ml	2	37.00
14	Coca-Cola 600ml	1	18.50
17	Coca-Cola 600ml	2	37.00
23	Coca-Cola 600ml	1	18.50
1	Sabritas Sal 45g	1	17.00
2	Sabritas Sal 45g	1	17.00
1	Sabritas Sal 45g	1	17.00
2	Sabritas Sal 45g	1	17.00
8	Sabritas Sal 45g	1	17.00
11	Sabritas Sal 45g	4	68.00
17	Sabritas Sal 45g	2	34.00
20	Sabritas Sal 45g	1	17.00
8	Gansito Marínela	1	15.50
14	Gansito Marínela	1	15.50
19	Gansito Marínela	1	15.50
4	Leche Entera 1L	5	130.00
4	Leche Entera 1L	5	130.00
9	Leche Entera 1L	1	26.00
15	Leche Entera 1L	2	52.00
21	Leche Entera 1L	2	52.00
4	Pan Blanco Grande	2	90.00

Consulta todos los pagos y a qué venta corresponde.

Sentencia o Comando utilizado/a:

```
SELECT Venta.idVenta, MetodoPago.NombreMetodo, Venta.Total
FROM Venta INNER JOIN MetodoPago
ON Venta.idMetodoPago=MetodoPago.idMetodoPago;
```

Resultado:

Tabla 12: Consulta todos los pagos y a qué venta corresponde.

idVenta	NombreMetodo	Total
1	Efectivo	35.50
3	Efectivo	18.50
5	Efectivo	50.00
6	Efectivo	35.50
8	Efectivo	18.50
10	Efectivo	50.00
11	Efectivo	44.00
13	Efectivo	44.50
15	Efectivo	87.00
18	Efectivo	12.00
19	Efectivo	34.00
21	Efectivo	37.00
22	Efectivo	71.00
24	Efectivo	37.50
27	Efectivo	12.00
28	Efectivo	18.50
2	Tarjeta de Débito	115.00
7	Tarjeta de Débito	115.00
12	Tarjeta de Débito	133.00
17	Tarjeta de Débito	67.00
20	Tarjeta de Débito	52.00
23	Tarjeta de Débito	35.00
26	Tarjeta de Débito	142.00
4	Tarjeta de Crédito	220.00
9	Tarjeta de Crédito	220.00

Consulta todas las ventas que corresponden a un cliente en particular y que cajero lo atendió.

Sentencia o Comando utilizado/a:

```
SELECT Venta.idVenta, Cliente.Nombre, Usuario.Nombre FROM Venta INNER
JOIN Cliente ON Venta.idCliente = Cliente.idCliente
INNER JOIN Usuario ON Venta.idUsuario = Usuario.idUsuario
WHERE Cliente.idCliente = 2;
```

Resultado:

Tabla 13: Consulta todas las ventas que corresponden a un cliente en particular y que cajero lo atendió

idVenta	Nombre	Nombre
2	Luis	Juan Pérez
7	Luis	Juan Pérez
28	Luis	Elena Torres

Consulta todos los clientes que fueron atendidos por un cajero.

Sentencia o Comando utilizado/a:

```
SELECT Venta.idVenta, Cliente.Nombre, Usuario.Nombre FROM Venta INNER
JOIN Cliente ON Venta.idCliente = Cliente.idCliente
INNER JOIN Usuario ON Venta.idUsuario = Usuario.idUsuario
WHERE Cliente.idCliente = 2;
```

Resultado:

Tabla 14: Consulta todos los clientes que fueron atendidos por un cajero.

Nombre	Apellido	Nombre
Andres	Caicedo	Juan Pérez
Luis	Hernández	Juan Pérez
Andres	Caicedo	Juan Pérez
Andres	Caicedo	Juan Pérez
Luis	Hernández	Juan Pérez
Andres	Caicedo	Juan Pérez
Joaquín	Luna	Juan Pérez
Camila	Morales	Juan Pérez

Plan de pruebas unitarias.

Define qué métodos o funciones serán probados de forma aislada, con sus entradas y salidas esperadas. Garantiza que cada componente del sistema funcione correctamente de manera independiente.

Usuario

Tabla 15: Plan de pruebas unitarias - Usuario.

Caso de Éxito	Clase	Metodo	Caso de Prueba	Resultado Esperado
	UsuarioRegistrado	SetNombre(nombre)	Andres, joshua	Se guarda el resultado
	UsuarioRegistrado	SetContraseña(contraseña)	1234, Andres123, joshua12!	Se guarda el resultado
	UsuarioRegistrado	SetIdTipoUsuario(idTipoUsuario)	11112222, 12345678	Se guarda el resultado

Caso de Excepcion	Clase	Metodo	Caso de Prueba	Resultado Esperado
	UsuarioRegistrado	SetNombre(nombre)	1234, . \$ % & & / (()	No se guarda el resultado
	UsuarioRegistrado	SetContraseña(contraseña)	andres1234!¿?&(Muchos caracteres)	No se guarda el resultado
	UsuarioRegistrado	SetIdTipoUsuario(idTipoUsuario)	11111andres1111, joshua222\$\$\$\$	No se guarda el resultado

Producto

Tabla 16: Plan de pruebas unitarias - Producto

SetNombre(nombre, descripcion)	1234, ·\$%&&/((()	No se guarda el resultado
SetPrecio(precio, stock)	48,00, veinte	No se guarda el resultado
SetIdProducto(idProducto)	cheetos1111	No se guarda el resultado

Metodo	Caso de Prueba	Resultado Esperado
GetNombre(nombre, descripcion)	Cheetos, cheetos sabor maiz	Se muestra el registro
GetPrecio(precio, stock)	35\$, 20 (disponibles)	Se muestra el registro
GetIdProducto(idProducto)	11112222, 12345678	Se muestra el registro

Metodo	Caso de Prueba	Resultado Esperado
GetNombre(nombre, descripcion)	1234, ·\$%&&/((()	No se muestra el registro
GetPrecio(precio, stock)	48,00, veinte	No se muestra el registro
GetIdProducto(idProducto)	cheetos1111	No se muestra el registro

Metodo	Caso de Prueba	Resultado Esperado
UpdateNombre(nombre, descripcion)	Cheetos Flamin, cheetos sabor maiz picoso	Se muestra el registro
UpdatePrecio(precio, stock)	35\$, 20 (disponibles)	Se muestra el registro

Metodo	Caso de Prueba	Resultado Esperado
UpdateNombre(nombre, descripcion)	1234, ·\$%&&/((()	No se muestra el registro
UpdatePrecio(precio, stock)	48,00, veinte	No se muestra el registro

Metodo	Caso de Prueba	Resultado Esperado
DeleteldProducto(idProducto)	12345678	Se elimina el registro

Metodo	Caso de Prueba	Resultado Esperado
DeleteldProducto(idProducto)	12%34/, ·\$%&&/((()	No se elimina el registro

Ventas

Tabla 17: Plan de pruebas unitarias - Ventas

Caso de Éxito	Clase	Metodo	Caso de Prueba	Resultado Esperado
	VentaRegistrada	SetTotal(total)	2300.50	Se guarda el resultado
	VentaRegistrada	SetFecha(fecha, hora)	25/06/2026 16:00	Se guarda el resultado
	VentaRegistrada	SetIdVenta(idVenta)	11112222, 12345678	Se guarda el resultado
	VentaRegistrada	SetIdMetodoPago(idMetodoPago)	11112222, 12345678	Se guarda el resultado
	VentaRegistrada	SetIdCliente(idCliente)	11112222, 12345678	Se guarda el resultado
	VentaRegistrada	SetIdUsuario(idUsuario)	11112222, 12345678	Se guarda el resultado

Caso de Excepcion	Clase	Metodo	Caso de Prueba	Resultado Esperado
	VentaRegistrada	SetTotal(total)	2350.6	No se guarda el resultado
	VentaRegistrada	SetFecha(fecha, hora)	30-02-2026 3:65	No se guarda el resultado
	VentaRegistrada	SetIdVenta(idVenta)	venta1234	No se guarda el resultado
	VentaRegistrada	SetIdMetodoPago(idMetodoPago)	metodo1111	No se guarda el resultado
	VentaRegistrada	SetIdCliente(idCliente)	cliente1111	No se guarda el resultado
	VentaRegistrada	SetIdUsuario(idUsuario)	usuario1111	No se guarda el resultado

Caso de Éxito	Clase	Metodo	Caso de Prueba	Resultado Esperado
---------------	-------	--------	----------------	--------------------

total
fecha
hora
idMetodoPago
idVenta
idCliente
idUsuario

UABC_FCQI_ANALISIS_Y_DISEÑO_DE_SISTEMAS.

VentaConsultada	GetTotal(total)	2300.50	Se muestra el registro
VentaConsultada	GetFecha(fecha, hora)	25/06/2026 16:00	Se muestra el registro
VentaConsultada	GetIdVenta(idVenta)	11112222, 12345678	Se muestra el registro
VentaConsultada	GetIdMetodoPago(idMetodoPago)	11112222, 12345678	Se muestra el registro
VentaConsultada	GetIdCliente(idCliente)	11112222, 12345678	Se muestra el registro
VentaConsultada	GetIdUsuario(idUsuario)	11112222, 12345678	Se muestra el registro

Caso de Excepcion	Clase	Metodo	Caso de Prueba	Resultado Esperado
	VentaConsultada	GetTotal(total)	2350.65	No se muestra el registro
	VentaConsultada	GetFecha(fecha, hora)	30-02-2026 3:00	No se muestra el registro
	VentaConsultada	GetIdVenta(idVenta)	venta1234	No se muestra el registro
	VentaConsultada	GetIdMetodoPago(idMetodoPago)	metodo1111	No se muestra el registro
	VentaConsultada	GetIdCliente(idCliente)	cliente1111	No se muestra el registro
	VentaConsultada	GetIdUsuario(idUsuario)	usuario1111	No se muestra el registro

Caso de Éxito	Clase	Metodo	Caso de Prueba	Resultado Esperado
	VentaModificada	UpdateTotal(total)	2370.50	Se modifica el registro
	VentaModificada	UpdateFecha(fecha, hora)	25/07/2026 18:00	Se modifica el registro

UABC_FCQI_ANALISIS_Y_DISEÑO_DE_SISTEMAS.

Caso de Excepcion	Clase	Metodo	Caso de Prueba	Resultado Esperado
	VentaModificada	UpdateTotal(total)	2350.85	No se modifica el registro
	VentaModificada	UpdateFecha(fecha, hora)	30-02-2026 4:00	No se modifica el registro

Caso de Éxito	Clase	Metodo	Caso de Prueba	Resultado Esperado
	VentaEliminada	DeleteldVenta(idVenta)	12345678	Se elimina el registro

Caso de Excepcion	Clase	Metodo	Caso de Prueba	Resultado Esperado
	VentaEliminada	DeleteldVenta(idVenta)	12%34/, ·\$%&&/((	No se elimina el registro

Método de pago

Tabla 18: Plan de pruebas unitarias - método pago.

Clase	Metodo	Caso de Prueba	Resultado Esperado
MetodoPagoRegistrado	SetNombre(nombre)	Tarjeta, Efectivo	Se guarda el resultado
MetodoPagoRegistrado	SetMonto(monto)	2350.50	Se guarda el resultado
MetodoPagoRegistrado	SetIdMetodoPago(idMetodoPago)	12345678	Se guarda el resultado

Clase	Metodo	Caso de Prueba	Resultado Esperado
MetodoPagoRegistrado	SetNombre(nombre)	1234, ·\$%&&/((	No se guarda el resultado
MetodoPagoRegistrado	SetMonto(monto)	2350.65	No se guarda el resultado
MetodoPagoRegistrado	SetIdMetodoPago(idMetodoPago)	metodopago1234	No se guarda el resultado

Pruebas de caja negra.

Evalúan el comportamiento del sistema desde la perspectiva del usuario, sin conocer el código interno. Verifican que ante determinadas entradas el sistema produce las salidas y mensajes esperados.

Casos de prueba

Tabla 19: PCN Plan de pruebas.

Plan de Pruebas de Caja Negra - Sistema POS		
#	Escenario / Caso de Prueba	Resultado Esperado (Salida)
1	Iniciar sesión con usuario 'admin' y contraseña correcta	Ingreso exitoso al panel de administración
2	Iniciar sesión con usuario 'admin' y contraseña incorrecta	Mensaje de error: 'Contraseña incorrecta'
3	Registrar un producto con precio de \$0.00	Mensaje de error: 'El precio debe ser mayor a 0'
4	Registrar un producto con nombre vacío	Mensaje de error: 'El nombre es obligatorio'
5	Agregar 10 unidades al carrito cuando solo hay 5 en stock	Mensaje de error: 'Stock insuficiente'
6	Realizar una venta y pagar con un método no registrado	Transacción rechazada
7	Consultar un producto por un ID que no existe	Mensaje: 'Producto no encontrado'
8	Finalizar venta con carrito lleno y pago en efectivo	Ticket generado y stock actualizado
9	Eliminar un usuario administrador del sistema	Confirmación de baja exitosa
10	Modificar stock de producto con valor negativo que deja el total bajo cero	Error: 'El ajuste dejaría el stock en negativo'
11	Intentar registrar un producto con nombre duplicado	Mensaje de error: 'El producto ya existe en el sistema'
12	Aplicar un descuento mayor al 100% sobre el total de la venta	Mensaje de error: 'El descuento no puede superar el total de la venta'
13	Generar reporte de ventas con rango de fechas donde fecha inicio es mayor a fecha fin	Mensaje de error: 'El rango de fechas no es válido'

Tablas de decisión

Tabla 20: Tabla de Decisión: Inicio de Sesión.

Tabla de Decisión: Inicio de Sesión				
Condiciones	R1	R2	R3	R4
El usuario existe en la BD	V	V	F	F
La contraseña es correcta	V	F	V	F
Resultados Esperados	R1	R2	R3	R4
Permitir acceso al sistema	X			
Mostrar error: Credenciales inválidas		X	X	X

Tabla 21: Tabla de Decisión: Finalizar Venta.

Tabla de Decisión: Finalizar Venta				
Condiciones	R1	R2	R3	R4
Hay productos en el carrito	V	V	F	F
Método de pago es válido (Efectivo/Tarjeta)	V	F	V	F
Resultados Esperados	R1	R2	R3	R4
Generar ticket y registrar venta	X			
Mostrar error: Carrito vacío o pago inválido		X	X	X

Tabla 22: Tabla de Decisión: Registrar Producto.

Tabla de Decisión: Registrar Producto				
Condiciones	R1	R2	R3	R4
El nombre del producto es válido (no vacío/duplicado)	V	V	F	F
El precio es mayor a \$0.00	V	F	V	F
Resultados Esperados	R1	R2	R3	R4
Producto registrado exitosamente	X			
Mostrar error: 'El precio debe ser mayor a 0'		X		
Mostrar error: 'El nombre es obligatorio o duplicado'			X	X

Tabla 23: Tabla de Decisión: Aplicar Descuento.

Tabla de Decisión: Aplicar Descuento				
Condiciones	R1	R2	R3	R4
El carrito tiene al menos un producto	V	V	F	F
El descuento es menor o igual al total	V	F	V	F
Resultados Esperados	R1	R2	R3	R4
Descuento aplicado y total actualizado	X			
Mostrar error: 'El descuento no puede superar el total'		X		
Mostrar error: 'No hay productos en el carrito'			X	X

Tabla 24: Tabla de Decisión: Generar Reporte de Ventas.

Tabla de Decisión: Generar Reporte de Ventas				
Condiciones	R1	R2	R3	R4
El rango de fechas es válido (inicio <= fin)	V	V	F	F
Existen ventas en el rango indicado	V	F	V	F

Resultados Esperados	R1	R2	R3	R4
Reporte generado con datos de ventas	X			
Mostrar mensaje: 'No hay ventas en el período seleccionado'		X		
Mostrar error: 'El rango de fechas no es válido'			X	X

Pruebas de caja blanca.

Analizan la lógica interna del código, revisando rutas de ejecución, condiciones y ciclos. Permiten detectar errores en la implementación que no serían visibles desde la interfaz.

Administrador

Tabla 25: PCB Administrador – Iniciar sesión.

Metodo: IniciarSesion(nombre, contrasenia)		
Pseudocodigo:		
<pre> Public boolean iniciarSesion(nombre, pass){ Boolean acceso = false; if(nombre != vacío AND pass != vacío){ usuario = buscarUsuario(nombre); if(usuario existe){ if(usuario.pass == pass){ acceso = true; } } } return acceso; } </pre>		
#	Caso de prueba	Resultado Esperado
1	Nombre y contraseña vacios	Falso
2	Nombre valido pero usuario inexistente	Falso
3	Usuario existente pero contraseña incorrecta	Falso
4	Usuario existente y contraseña correcta	Verdadero

Tabla 26: PCB Administrador – Alta usuario

Método: altaUsuario(nombre, rol, pass)		
Pseudocódigo:		
<pre> Public void altaUsuario(nombre, rol, pass){ if(nombre != vacio AND rol != vacio AND pass != vacio){ if(buscarUsuario(nombre) == null){ Usuario nuevo = new Usuario(nombre, rol, pass); guardarUsuario(nuevo); imprimir("Usuario creado con éxito"); } else { imprimir("El nombre de usuario ya existe"); } } else { imprimir("Faltan datos obligatorios"); } } </pre>		
#	Caso de prueba	Resultado Esperado
1	Nombre, rol o pass están vacíos o nulos	Falso
2	Datos completos, pero el nombre ya existe	Falso
3	Datos completos y el nombre es nuevo	Verdadero

Tabla 27: : PCB Administrador – Modificar usuario.

Método: modificarUsuario(idUsuario, datos)		
Pseudocódigo:		
<pre> Public void modificarUsuario(idUsuario, datos){ if(idUsuario > 0){ if(datos.nombre != vacio AND datos.rol != vacio){ Usuario u = consultarUsuario(idUsuario); if(u != null){ actualizarEnBD(idUsuario, datos); imprimir("Usuario actualizado exitosamente"); } else { imprimir("Error: El usuario no existe"); } } else { imprimir("Error: Los datos no pueden estar vacíos"); } } else { imprimir("ID de usuario inválido"); } } </pre>		
#	Caso de prueba	Resultado Esperado
1	idUsuario es 0 o negativo	Falso
2	idUsuario válido pero datos.nombre o datos.rol están vacíos	Falso
3	Datos completos pero el idUsuario no existe	Falso
4	Datos completos y el usuario existe	Verdadero

Tabla 28: PCB Administrador – Baja usuario]

Método: bajaUsuario(idUsuario)		
Pseudocódigo:		
<pre> Public boolean bajaUsuario(idUsuario){ boolean exito = false; if(idUsuario > 0){ Usuario u = consultarUsuario(idUsuario); if(u != null){ eliminarDeBD(idUsuario); exito = true; } } return exito; } </pre>		
#	Caso de prueba	Resultado Esperado
1	idUsuario es 0 o negativo	Falso
2	idUsuario válido pero no existe	Falso
3	idUsuario válido y existe	Verdadero

Tabla 29: PCB Administrador – Alta cliente

Método: altaCliente(nombre, telefono)		
Pseudocodigo:		
<pre> Public void altaCliente(nombre, telefono){ if(nombre != vacio){ if(telefono != vacio AND longitud(telefono) == 10){ guardarCliente(nombre, telefono); imprimir("Cliente registrado"); } else { imprimir("Telefono inválido"); } } else { imprimir("El nombre es obligatorio"); } } </pre>		
#	Caso de prueba	Resultado Esperado
1	nombre está vacío	Falso
2	nombre válido, pero telefono vacío o longitud distinta a 10	Falso
3	nombre válido y telefono de 10 digitos	Verdadero

Tabla 30: PCB Administrador – Consultar usuario.

Método: consultarUsuario(idUsuario)		
Pseudocodigo:		
<pre> Public Usuario consultarUsuario(idUsuario){ Usuario u = null; if(idUsuario > 0){ u = buscarUsuarioBD(idUsuario); } return u; // Retorna nulo si no lo encuentra o si el ID es inválido } </pre>		
#	Caso de prueba	Resultado Esperado
1	idUsuario es 0 o negativo	Falso
2	idUsuario válido pero no existe	Falso
3	idUsuario válido y existe	Verdadero

Tabla 31: PCB Administrador – Modificar cliente.

Método: modificarCliente(idCliente, datos)		
Pseudocodigo:		
<pre> Public void modificarCliente(idCliente, datos){ if(idCliente > 0){ if(datos.nombre != vacio AND longitud(datos.telefono) == 10){ Cliente c = consultarCliente(idCliente); if(c != null){ actualizarClienteBD(idCliente, datos); imprimir("Cliente actualizado"); } else { imprimir("El cliente no existe"); } } else { imprimir("Datos inválidos (Nombre vacío o Telefono incorrecto)"); } } else { imprimir("ID de cliente inválido"); } } </pre>		
#	Caso de prueba	Resultado Esperado
1	idCliente es 0 o negativo	Falso
2	idCliente válido pero datos.nombre vacío o telefono mayor a 10 digitos	Falso
3	Datos correctos pero el cliente no existe	Falso
4	Datos correctos y cliente existe	Verdadero

Tabla 32: PCB Administrador – consultar cliente.

Método: consultarCliente(idCliente)		
Pseudocódigo:		
<pre> Public Cliente consultarCliente(idCliente){ Cliente c = null; if(idCliente > 0){ c = buscarClienteBD(idCliente); } return c; } </pre>		
#	Caso de prueba	Resultado Esperado
1	idCliente es 0 o negativo	Falso
2	idCliente válido pero no existe en BD	Falso
3	idCliente válido y existe en BD	Verdadero

Cajero

Tabla 33: PCB cajero – Iniciar sesión.

Metodo: IniciarSesion(nombre, contrasenia)		
Pseudocódigo:		
<pre> Public boolean iniciarSesion(nombre, pass){ Boolean acceso = false; if(nombre != vacío AND pass != vacío){ usuario = buscarUsuario(nombre); if(usuario existe){ if(usuario.pass == pass){ acceso = true; } } } return acceso; } </pre>		
#	Caso de prueba	Resultado Esperado
1	Nombre y contraseña vacios	Falso
2	Nombre valido pero usuario inexistente	Falso
3	Usuario existente pero contraseña incorrecta	Falso
4	Usuario existente y contraseña correcta	Verdadero

Tabla 34: PCB cajero – Seleccionar producto.

Método: seleccionarProducto(idProducto, cantidad)		
Pseudocódigo:		
<pre> Public void seleccionarProducto(idProducto, cantidad){ if(idProducto > 0 AND cantidad > 0){ int stockDisponible = consultarInventarioLimitado(idProducto); if(stockDisponible != -1){ // -1 si no existe if(stockDisponible >= cantidad){ agregarAlCarrito(idProducto, cantidad); imprimir("Producto agregado"); } else { imprimir("Stock insuficiente"); } } else { imprimir("Producto no encontrado"); } } else { imprimir("Cantidad o ID inválidos"); } } </pre>		
#	Caso de prueba	Resultado Esperado
1	idProducto o cantidad es 0 o menor	Falso
2	Datos válidos, pero el producto no existe	Falso
3	Producto existe, pero cantidad solicitada es mayor al stock	Falso
4	Producto existe y cantidad solicitada es menor o igual al stock	Verdadero

Tabla 35: PCB cajero – Consultar precio.

Método: consultarPrecio(idProducto)		
Pseudocódigo:		
<pre> Public double consultarPrecio(idProducto){ double precio = -1.0; // Valor por defecto para error if(idProducto > 0){ Producto p = buscarProductoBD(idProducto); if(p != null){ precio = p.precio; } } return precio; } </pre>		
#	Caso de prueba	Resultado Esperado
1	idProducto es 0 o negativo	Falso
2	idProducto válido pero el producto no existe	Falso
3	idProducto válido y producto existe	Verdadero

Tabla 36: PCB cajero – Registrar venta.

Método: registrarVenta()		
Pseudocódigo:		
<pre> Public void registrarVenta(){ if(carrito.obtenerTotalItems() > 0){ int idVentaNueva = guardarVentaBD(carrito.total, carrito.idMetodoPago); if(idVentaNueva > 0){ for(Item item : carrito.items){ guardarDetalleVentaBD(idVentaNueva, item.idProducto, item.cantidad); descontarStockBD(item.idProducto, item.cantidad); } imprimir("Venta registrada correctamente en BD"); } else { imprimir("Error al conectar con la Base de Datos"); } } else { imprimir("Error: No se puede registrar una venta sin productos"); } } </pre>		
#	Caso de prueba	Resultado Esperado
1	El carrito de compras está vacío (0 items)	Falso
2	Carrito con productos pero la BD falla al guardar la Venta	Falso
3	Carrito con productos y la BD guarda correctamente	Verdadero

Tabla 37: PCB cajero – Finalizar transacción

Método: finalizarTransaccion(metodoPago)		
Pseudocódigo:		
<pre> Public boolean finalizarTransaccion(metodoPago){ boolean transaccionExitosa = false; if(carrito.obtenerTotalItems() > 0){ if(metodoPago == "Efectivo" OR metodoPago == "Tarjeta"){ registrarVenta(); vaciarCarrito(); transaccionExitosa = true; } } return transaccionExitosa; } </pre>		
#	Caso de prueba	Resultado Esperado
1	El carrito de compras está vacío	Falso
2	Carrito con productos, pero metodoPago no reconocido	Falso
3	Carrito con productos y metodoPago válido	Verdadero

Tabla 38: PCB cajero – Generar tiquet.

Método: generarTicket(idVenta)		
Pseudocódigo:		
<pre> Public String generarTicket(idVenta){ String ticket = ""; if(idVenta > 0){ Venta v = consultarVentaBD(idVenta); if(v != null){ ticket = "TICKET # " + v.id + "\nTotal: \$" + v.total; } else { ticket = "Error: Venta no encontrada"; } } else { ticket = "Error: ID de venta inválido"; } return ticket; } </pre>		
#	Caso de prueba	Resultado Esperado
1	idVenta negativo o 0	Falso
2	idVenta válido, pero no existe registro de esa venta	Falso
3	idVenta válido y existe	Verdadero

Cliente

Tabla 39: PCB cliente – Realizar pago.

Método: realizarPago(monto)		
Pseudocodigo:		
<pre> Public boolean realizarPago(double monto){ boolean pagoExitoso = false; double totalAPagar = this.carrito.obtenerTotal(); if(totalAPagar > 0){ if(monto > 0){ if(monto >= totalAPagar){ double cambio = monto - totalAPagar; imprimir("Pago exitoso. Cambio: \$" + cambio); pagoExitoso = true; } else { imprimir("Error: Fondos insuficientes. Falta dinero."); } } else { imprimir("Error: El monto ingresado no es válido."); } } else { imprimir("Error: No hay un saldo a pagar."); } return pagoExitoso; } </pre>		
#	Caso de prueba	Resultado Esperado
1	Total, a pagar es 0	Falso
2	monto entregado es <= 0	Falso
3	monto es menor al total a pagar	Falso
4	monto es mayor o igual al total a pagar	Verdadero

Tabla 40: PCB cliente – Añadir.

Método: añadirTips(propina)		
Pseudocodigo:		
<pre> Public void añadirTips(double propina){ if(propina >= 0){ double nuevoTotal = this.carrito.obtenerTotal() + propina; this.carrito.actualizarTotal(nuevoTotal); imprimir("Propina añadida de: \$" + propina); } else { imprimir("Error: La propina no puede ser un valor negativo"); } } </pre>		
#	Caso de prueba	Resultado Esperado
1	propina es un valor negativo (ej. -10)	Falso
2	propina es 0	Falso
3	propina es mayor a 0 (ej. 15.50)	Verdadero

Tabla 41: PCB cliente – Solicitar productos.

Método: solicitarProductos(idProducto)		
Pseudocodigo:		
<pre> Public Producto solicitarProductos(int idProducto){ Producto p = null; if(idProducto > 0){ p = consultarInventarioGeneral(idProducto); if(p == null){ imprimir("El producto solicitado no está disponible"); } } else { imprimir("Código de producto inválido"); } return p; } </pre>		
#	Caso de prueba	Resultado Esperado
1	idProducto es <= 0	Falso
2	idProducto es válido pero no hay stock/no existe	Falso
3	idProducto válido y existe	Verdadero

Tabla 42: PCB cliente – Carrito de productos.

Método: carritoDeProductos(idVenta)		
Pseudocodigo:		
<pre> Public DetalleVenta carritoDeProductos(int idVenta){ DetalleVenta detalle = null; if(idVenta > 0){ detalle = buscarDetalleVentaBD(idVenta); if(detalle != null){ imprimir("Mostrando carrito de productos..."); } else { imprimir("El carrito está vacío o la venta no existe"); } } else { imprimir("ID de venta/carrito inválido"); } return detalle; } </pre>		
#	Caso de prueba	Resultado Esperado
1	idVenta es 0 o negativo	Falso
2	idVenta válido, pero no tiene productos registrados	Falso
3	idVenta válido y tiene productos	Verdadero

Tabla 43: PCB cliente – Realizar pago.

Método: realizarPago(monto)		
Pseudocodigo:		
<pre> Public void solicitarTicket(int idVenta){ if(idVenta > 0){ Venta v = consultarVentaBD(idVenta); if(v != null){ if(v.estado == "Pagada"){ imprimir("--- TICKET DE COMPRA ---"); imprimir("Total: \$" + v.total); } else { imprimir("Error: La venta aún no ha sido pagada"); } } else { imprimir("Error: No se encontró la venta solicitada"); } } else { imprimir("Error: Número de ticket (ID) inválido"); } } </pre>		
#	Caso de prueba	Resultado Esperado
1	idVenta es 0 o negativo	Falso
2	idVenta válido pero la venta no existe en la BD	Falso
3	La venta existe, pero no está pagada.	Falso
4	La venta existe y ya está pagada	Verdadero

Venta

Tabla 44: PCB ventas – reporte ventas.

Método: reporteVentas(fechaInicio, fechaFin)		
Pseudocódigo:		
<pre> Public Reporte reporteVentas(Date fechaInicio, Date fechaFin){ Reporte rep = null; if(fechaInicio != null AND fechaFin != null){ if(fechaInicio <= fechaFin){ List<Venta> lista = buscarVentasPorFechaBD(fechaInicio, fechaFin); if(lista.size() > 0){ rep = new Reporte(lista); imprimir("Reporte generado con éxito"); } else { imprimir("No hay ventas en este rango de fechas"); } } else { imprimir("Error: La fecha de inicio no puede ser mayor a la fecha de fin"); } } else { imprimir("Error: Las fechas no pueden estar vacías"); } return rep; } </pre>		
#	Caso de prueba	Resultado Esperado
1	fechaInicio o fechaFin son nulos	Falso
2	fechaInicio es una fecha mayor (después) que fechaFin	Falso
3	Fechas válidas, pero no hay ventas registradas en esos días	Falso
4	Fechas válidas y existen ventas en la base de datos	Verdadero

Tabla 45: PCB ventas – generar detalle venta.

Método: generarDetalleVenta(idProducto, cantidad)		
Pseudocódigo:		
<pre> Public DetalleVenta generarDetalleVenta(idProducto, cantidad){ DetalleVenta detalle = null; if(idProducto > 0 AND cantidad > 0){ Producto p = consultarProducto(idProducto); if(p != null){ detalle = new DetalleVenta(); detalle.idProducto = p.id; detalle.nombre = p.nombre; detalle.cantidad = cantidad; detalle.subTotal = p.precio * cantidad; } else { imprimir("Error: Producto no encontrado"); } } else { imprimir("Error: Datos inválidos para el detalle"); } return detalle; } </pre>		
#	Caso de prueba	Resultado Esperado
1	idProducto o cantidad son 0 o negativos	Falso
2	Datos válidos, pero el producto no existe en BD	Falso
3	Datos válidos y el producto existe	Verdadero

Tabla 46: PCB ventas – modificar venta.

Método: modificarVenta(idVenta, nuevosDatos)		
Pseudocódigo:		
<pre> Public void modificarVenta(idVenta, nuevosDatos){ if(idVenta > 0){ if(nuevosDatos != null){ Venta v = consultarVentaBD(idVenta); if(v != null){ actualizarVentaBD(idVenta, nuevosDatos); imprimir("Venta modificada con éxito"); } else { imprimir("Error: La venta no existe"); } } else { imprimir("Error: Los datos a modificar están vacíos"); } } else { imprimir("Error: ID de venta inválido"); } } </pre>		
#	Caso de prueba	Resultado Esperado
1	idVenta es 0 o negativo	Falso
2	idVenta válido pero nuevosDatos es nulo	Falso
3	Datos válidos, pero idVenta no existe en BD	Falso
4	Datos válidos y la venta existe	Verdadero

Producto

Tabla 47: PCB producto – alta productos.

Método: altaProducto(nombre, precio, stock)		
Pseudocódigo:		
<pre> Public void altaProducto(nombre, precio, stock){ if(nombre != vacio){ if(precio > 0){ if(stock >= 0){ Producto p = new Producto(nombre, precio, stock); guardarProductoBD(p); imprimir("Producto registrado con éxito"); } else { imprimir("Error: El stock inicial no puede ser negativo"); } } else { imprimir("Error: El precio debe ser mayor a 0"); } } else { imprimir("Error: El nombre del producto es obligatorio"); } } </pre>		
#	Caso de prueba	Resultado Esperado
1	nombre está vacío	Falso
2	nombre válido, pero precio es 0 o negativo	Falso
3	nombre y precio válidos, pero stock es negativo	Falso
4	nombre, precio y stock validos	Verdadero

Tabla 48: PCB producto – Modificar producto.

Método: modificarProducto(idProducto, nuevosDatos)		
Pseudocódigo:		
<pre> Public boolean modificarProducto(idProducto, nuevosDatos){ boolean exito = false; if(idProducto > 0){ if(nuevosDatos.nombre != vacio AND nuevosDatos.precio > 0 AND nuevosDatos.stock >= 0){ Producto p = consultarProducto(idProducto); if(p != null){ actualizarProductoBD(idProducto, nuevosDatos); exito = true; } } } return exito; } </pre>		
#	Caso de prueba	Resultado Esperado
1	idProducto es 0 o negativo	Falso
2	idProducto válido, pero nuevosDatos tiene errores (nombre vacío, precio <= 0 o stock < 0)	Falso
3	Datos válidos, pero el idProducto no existe	Falso
4	Datos válidos y el producto existe	Verdadero

Tabla 49: PCB producto – consultar producto.

Método: consultarProducto(idProducto)		
Pseudocódigo:		
<pre> Public Producto consultarProducto(idProducto){ Producto p = null; if(idProducto > 0){ p = buscarProductoBD(idProducto); } return p; } </pre>		
#	Caso de prueba	Resultado Esperado
1	idProducto es 0 o negativo	Falso
2	idProducto válido pero no existe	Falso
3	idProducto válido y existe	Verdadero

Tabla 50: PCB producto – Ajustar stock

Método: ajustarStock(idProducto, cantidad)		
Pseudocódigo:		
<pre> Public void ajustarStock(idProducto, cantidad){ if(idProducto > 0){ Producto p = consultarProducto(idProducto); if(p != null){ int nuevoStock = p.stock + cantidad; // cantidad puede ser positiva o negativa if(nuevoStock >= 0){ actualizarStockBD(idProducto, nuevoStock); imprimir("Stock actualizado correctamente"); } else { imprimir("Error: El ajuste dejaría el stock en números negativos"); } } else { imprimir("Error: Producto no encontrado"); } } else { imprimir("Error: ID de producto inválido"); } } </pre>		
#	Caso de prueba	Resultado Esperado
1	idProducto es 0 o negativo	Falso
2	idProducto válido, pero el producto no existe en BD	Falso
3	Producto existe, pero la cantidad restada al stock actual (ej. 5) da un resultado negativo	Falso
4	Producto existe y la suma/resta de la cantidad deja un stock válido (>= 0)	Verdadero

5 Conclusiones

5.1 Reflexión sobre el desarrollo del proyecto.

El desarrollo de NexoPay representó un ejercicio integral de ingeniería de software, en el que los conceptos teóricos de la Programación Orientada a Objetos se tradujeron en decisiones concretas de diseño e implementación. La jerarquía de clases de usuarios, la separación del sistema en capas con responsabilidades bien definidas y la integración con una base de datos relacional son ejemplos tangibles de cómo los principios de POO facilitan la construcción de software organizado, legible y mantenible.

El proceso de análisis previo al desarrollo resultó fundamental: los diagramas UML elaborados permitieron anticipar problemas de diseño, definir con claridad las responsabilidades de cada componente y establecer un mapa de trabajo compartido para el equipo. Esta experiencia refuerza la importancia de dedicar tiempo al diseño antes de comenzar a codificar.

La integración de múltiples áreas del conocimiento, incluyendo bases de datos, interfaces gráficas, lógica de negocio y principios de diseño, demostró que el desarrollo de software real es una actividad multidisciplinaria que requiere no solo habilidades técnicas, sino también capacidad de planificación, comunicación y trabajo en equipo.

5.2 Lecciones aprendidas.

- **El diseño previo ahorra tiempo de desarrollo:** Los diagramas UML elaborados antes de comenzar a codificar actuaron como una guía clara que redujo la ambigüedad y el retrabajo durante la implementación.
- **La separación de responsabilidades facilita el mantenimiento:** Organizar el código en capas (vistas, controladores, servicios, modelos, datos) hizo que las modificaciones en una capa raramente afectaran a las demás, simplificando las correcciones y mejoras.
- **Los nombres descriptivos son más importantes de lo que parecen:** Invertir tiempo en elegir nombres claros para clases, métodos y variables redujo significativamente el tiempo necesario para comprender el código al revisarlo días después de haberlo escrito.
- **Las pruebas deben planificarse desde el inicio:** Definir los casos de prueba junto con los requerimientos permitió detectar casos límite (stock negativo, carrito vacío, credenciales inválidas) que de otro modo podrían haberse pasado por alto hasta etapas tardías del desarrollo.
- **La base de datos requiere diseño cuidadoso:** Las relaciones entre tablas, las restricciones de integridad referencial y la elección de los tipos de datos tuvieron un impacto directo en la facilidad de implementación de las consultas y en la consistencia de los datos.

5.3 Posibles mejoras futuras.

Con miras a versiones futuras del sistema, se identifican con las siguientes mejoras que ampliarían sus capacidades y su robustez:

- **Módulo de devoluciones:** Implementar la funcionalidad de procesar devoluciones de productos, con actualización automática del inventario y registro de la operación en el historial de ventas.
- **Gestión de descuentos y promociones:** Añadir soporte para descuentos por producto, por categoría o por monto total de la venta, incluyendo la configuración de vigencias y condiciones.
- **Reportes avanzados con gráficas:** Enriquecer el módulo de reportes con visualizaciones gráficas (barras, líneas) que faciliten el análisis de tendencias de ventas a lo largo del tiempo.
- **Soporte multi-sucursal:** Extender la arquitectura para soportar múltiples puntos de venta conectados a una base de datos centralizada, con sincronización de inventario en tiempo real.
- **Exportación de reportes a PDF:** Permitir la exportación de los reportes de ventas en formato PDF para su archivo y distribución.
- **Notificaciones de stock bajo:** Implementar alertas automáticas cuando el stock de un producto caiga por debajo de un umbral configurable, facilitando la gestión de reabastecimiento.
- **Migración a arquitectura cliente-servidor web:** En una versión más avanzada, migrar la interfaz de Swing a una interfaz web con JavaFX o tecnologías web estándar, permitiendo el acceso al sistema desde múltiples dispositivos.

6 Bibliografía

[1] K. C. Laudon and J. P. Laudon, *Sistemas de información gerencial*. Pearson, 2012.

[2] P. J. Deitel and H. M. Deitel, *Java: How to Program*. Pearson College Division, 2012.

[3] "Java Documentation - Get Started," Oracle Help Center, Jan. 31, 2023. [Online]. Available: <https://docs.oracle.com/en/java/>

[4] T. Babic, "20 ejemplos de consultas SQL básicas para principiantes," LearnSQL.es, Sep. 20, 2023. [Online]. Available: <https://learnsql.es/blog/20-ejemplos-de-consultas-sql-basicas-para-principiantes-una-vision-completa/>

[5] J. M. Alarcón, "TUTORIAL SQL #3: Consultas SELECT multi-tabla básicas - JOIN," campusMVP.es, Nov. 08, 2021. [Online]. Available:

<https://www.campusmvp.es/recursos/post/Fundamentos-de-SQL-Consultas-SELECT-multi-tabla-JOIN.aspx>

[6] "Arquitectura Monolítica," reactiveprogramming.io. [Online]. Available: <https://reactiveprogramming.io/blog/es/estilos-arquitectonicos/monolitico>

[7] Amazon Web Services, "¿Cuál es la diferencia entre la arquitectura monolítica y la de microservicios?," AWS. [Online]. Available: <https://aws.amazon.com/es/compare/the-difference-between-monolithic-and-microservices-architecture/>

[8] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.